

Grado en Ingeniería Informática
Curso académico 2025-2026

Trabajo Fin de Grado

“Optimización de bajo coste
computacional para la programación de
tareas industriales en PYMEs: Un
enfoque basado en Iterated Greedy para
el Permutation Flow-Shop”

Augusto De Pinho Gomes

Tutor

David Yagüe Cuevas

Leganés, septiembre de 2026



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

La programación de tareas en líneas de producción en serie se formaliza mediante el *Permutation Flow-Shop Scheduling Problem* (PFSP), un problema de optimización combinatoria NP-duro. La función objetivo que se optimiza es únicamente el *makespan*: el instante en que el último trabajo abandona la última máquina. Su resolución a escala industrial exige métodos aproximados, pues los exactos y los *solvers* comerciales resultan inviables o costosos, una barrera especialmente relevante para las pequeñas y medianas empresas (PYMEs). Este trabajo diseña, implementa y valida una solución de bajo coste computacional, ejecutable en equipos convencionales y sin dependencias comerciales, basada en la metaheurística *Iterated Greedy* (IG). Sobre ella se propone una aportación: un operador de destrucción inteligente que, en lugar de extraer trabajos al azar de toda la secuencia, restringe ese sorteo a los que más tiempo muerto (*idle time*) generan, mediante una lista restringida de candidatos y un muestreo uniforme.

La metodología parte de la heurística constructiva de Nawaz, Ensore y Ham (NEH) como línea base y construye el IG clásico de forma incremental, validando una aceleración del cálculo del *makespan* y calibrando el presupuesto de parada y el tamaño de la lista de candidatos. La evaluación es empírica y reproducible, con semillas fijas y datos crudos conservados, sobre las 600 instancias de dos conjuntos de referencia: las clásicas de Taillard, como prueba de concepto, y las de alta dureza de Vallada, Ruiz y Framinan (VRF). Se examinan los dos ejes que determinan su utilidad práctica: la calidad de las soluciones, medida con la desviación porcentual relativa (RPD) y contrastada con pruebas no paramétricas, y el coste de cómputo, que no forma parte de la función objetivo pero condiciona su viabilidad en un equipo convencional.

El IG mejora sustancialmente a la línea base: el RPD medio pasa del 3,47 % del NEH al 0,60 %. La destrucción guiada iguala en calidad a la aleatoria sobre el conjunto completo y la mejora en las instancias grandes y duras (las de mayor interés industrial) por un margen estrecho pero estadísticamente significativo, y alcanza esa calidad con un 4,5 % menos de tiempo a igual presupuesto, al ahorrar pasadas de búsqueda local. Una variante puramente voraz del operador resulta peor que el azar: el valor de la propuesta está en acotar la aleatoriedad, no en suprimirla. La solución se entrega como un producto mínimo viable reproducible, orientado a su adopción por PYMEs.

Palabras clave: PFSP; *Iterated Greedy*; *makespan*; metaheurísticas; PYME

AGRADECIMIENTOS

Quiero dar las gracias de todo corazón a mi familia, a mi novia, a mis amigos y a mis profesores por el apoyo recibido durante estos años de grado, y en particular durante los meses de elaboración de este trabajo. Su confianza incondicional es la razón de que haya llegado hasta aquí.

Agradezco también a mi tutor, David Yagüe Cuevas, la confianza que depositó en mí y el acompañamiento cercano con que ha guiado este trabajo: el conocimiento que compartió en cada una de nuestras reuniones y la orientación que me dio en ellas son buena parte de lo que ha permitido llevarlo a término.

Por último, a la Universidad Carlos III de Madrid, por haber sido un espacio de aprendizaje e inspiración durante unos años que han dejado huella no solo en lo académico y lo profesional, sino también en lo personal. Las experiencias que he vivido y las oportunidades que he tenido aquí me han enseñado a afrontar los desafíos y, sobre todo, a disfrutarlos.

ÍNDICE GENERAL

LISTA DE ACRÓNIMOS	xvi
1. INTRODUCCIÓN.	1
1.1. Planteamiento del problema.	1
1.2. Motivación y antecedentes	2
1.3. Objetivos	2
1.4. Estructura del documento	3
2. ESTADO DEL ARTE.	4
2.1. El problema del flow-shop de permutación y su intratabilidad.	4
2.2. Cálculo del <i>makespan</i>	5
2.3. El tiempo muerto y su descomposición	6
2.4. Panorámica de los métodos de resolución	7
2.5. Heurísticas constructivas: el algoritmo NEH	8
2.6. Iterated Greedy: el algoritmo de referencia	9
2.7. Evolución del Iterated Greedy (2014–2024).	10
2.8. El operador de destrucción: aleatorio frente a guiado.	10
2.9. Marco de evaluación: Taillard frente a VRF.	11
2.10. Síntesis comparativa y posicionamiento del trabajo	12
3. DISEÑO Y METODOLOGÍA	14
3.1. Arquitectura del software	14
3.2. Heurística constructiva NEH	15
3.3. Iterated Greedy (IG) clásico.	16
3.3.1. Destrucción aleatoria	16
3.3.2. Construcción (reinserción NEH)	16
3.3.3. Búsqueda local de inserción.	16
3.3.4. Aceptación tipo Simulated Annealing	17
3.3.5. Criterio de parada	17
3.3.6. Pseudocódigo del bucle IG	17

3.4. Destrucción inteligente (aportación)	19
3.4.1. Puntuación de tiempo muerto	19
3.4.2. Destrucción guiada por tiempo muerto.	20
3.4.3. Variante voraz y coste	21
3.5. Diseño experimental	21
3.5.1. Benchmarks.	21
3.5.2. Métrica de calidad	22
3.5.3. Réplicas y reproducibilidad	22
3.5.4. Runner de campaña	22
3.5.5. Entorno de experimentación.	22
4. RESULTADOS EXPERIMENTALES	24
4.1. Resumen de las campañas experimentales	24
4.2. Baseline NEH (EXP-001 y EXP-002)	24
4.2.1. Resultados sobre Taillard	26
4.2.2. Resultados sobre VRF	27
4.3. Calibración del presupuesto del IG (EXP-003)	27
4.4. Aceleración de la evaluación de inserciones (EXP-005)	29
4.5. Iterated Greedy (EXP-004)	31
4.5.1. Resultados globales	31
4.5.2. Monotonía y sanidad.	32
4.6. Destrucción inteligente	32
4.6.1. Calibración del parámetro α (EXP-006)	33
4.6.2. Comparación con la destrucción aleatoria	34
4.6.3. Coste de cómputo de los operadores	36
4.7. Análisis comparativo.	38
5. CONCLUSIONES Y TRABAJO FUTURO	40
5.1. Conclusiones	40
5.2. Limitaciones	42
5.3. Líneas de trabajo futuro	44
6. MARCO REGULADOR	46
6.1. Naturaleza del trabajo y alcance	46

6.2. Propiedad intelectual e industrial	46
6.2.1. Licencias del software de terceros	46
6.2.2. Licencia del código propio	47
6.2.3. Licencia de los conjuntos de instancias (<i>benchmarks</i>)	47
6.2.4. Licencia de la memoria	47
6.2.5. Patentabilidad.	48
6.3. Protección de datos personales	48
6.4. Estándares técnicos	49
6.5. Responsabilidad profesional y ética	49
6.6. Síntesis del marco regulador aplicable	49
7. ENTORNO SOCIO-ECONÓMICO Y PRESUPUESTO	50
7.1. Planificación temporal del proyecto	50
7.2. Presupuesto del TFG.	51
7.2.1. Costes de personal	51
7.2.2. Costes de hardware (amortización).	51
7.2.3. Costes de software	52
7.2.4. Costes indirectos y total	52
7.3. Impacto socio-económico	53
7.3.1. Impacto económico	53
7.3.2. Impacto social	54
7.3.3. Impacto medioambiental (cualitativo)	54
7.3.4. Impacto ético	54
7.4. Relación con los Objetivos de Desarrollo Sostenible	54
7.5. Síntesis del coste y del impacto esperado	55
BIBLIOGRAFÍA	56
ANEXO A. GUÍA DE REPRODUCCIÓN (MVP)	59
ANEXO B. DECLARACIÓN DE USO DE IA GENERATIVA	62

ÍNDICE DE FIGURAS

1.1	Ejemplo de <i>flow-shop</i> de permutación con tres máquinas y cuatro trabajos. El <i>makespan</i> (C_{max}) es el instante en que el último trabajo termina en la última máquina; en gris, los tiempos muertos (<i>idle</i>).	1
3.1	Paso de inserción de NEH: el trabajo entrante se prueba en todas las posiciones de la secuencia parcial y se fija en la que minimiza el <i>makespan</i> parcial (valores ilustrativos).	15
3.2	Ciclo del <i>Iterated Greedy</i> : a partir de una solución inicial NEH, se itera destrucción, reconstrucción codiciosa, búsqueda local y aceptación hasta agotar el presupuesto.	16
3.3	Destrucción aleatoria (izquierda) frente a destrucción inteligente (derecha): esta última restringe el sorteo a los trabajos responsables del mayor tiempo muerto en las máquinas críticas, preservando los bloques eficientes.	19
4.1	RPD medio del NEH por número de máquinas (EXP-001/002): Taillard (frente al <i>upper bound</i>) y VRF (frente al mejor <i>makespan</i> conocido). El RPD sube con fuerza al pasar de $m = 5$ a $m = 10$ en ambos <i>benchmarks</i> y se mantiene alto después, con el máximo de VRF en $m = 15$. Taillard no incluye instancias con $m \in \{15, 40, 60\}$, de ahí la ausencia de barra en esos valores.	28
4.2	Calibración del presupuesto del IG (EXP-003): RPD medio frente al presupuesto de iteraciones (escala logarítmica); la rodilla en 500 marca el compromiso calidad/coste adoptado.	29
4.3	Coste por ejecución frente al tamaño de la instancia a presupuesto 500 (EXP-005), implementación directa ($O(n^3m)$) frente a la acelerada ($O(n^2m)$); escala log-log. La divergencia de las curvas ilustra el cambio de complejidad.	31
4.4	RPD medio del NEH frente al IG por número de máquinas (EXP-004), separado por benchmark (Taillard, frente al UB; VRF, frente al mejor conocido). El IG reduce el RPD del <i>baseline</i> en todo el rango de m	32
4.5	Calibración de α (EXP-006): RPD medio frente al tamaño de la RCL. Descenso pronunciado hasta 0,30 y meseta posterior (0,30–0,50); se fija $\alpha = 0,30$ en la rodilla.	33

4.6	Eje eficiencia (EXP-007): convergencia del RPD medio frente al presupuesto para la destrucción aleatoria, la voraz (<i>idle-greedy</i>) y la guiada (<i>idle-rcl</i> , $\alpha = 0,30$) sobre el subconjunto de calibración.	35
4.7	Origen del ahorro de tiempo de la destrucción guiada (EXP-008, medición pareada sobre 36 instancias con tres semillas y presupuesto de 500 iteraciones). Izquierda: las tres reducciones frente a la destrucción aleatoria —pasadas de búsqueda local, evaluaciones de inserción y tiempo de ejecución— coinciden en torno al 5 %. Derecha: relación entre el ahorro de trabajo y el de tiempo en cada instancia; la diagonal discontinua representa la igualdad entre ambos.	38
4.8	Comparación de calidad (EXP-007, 600 instancias). Izquierda: RPD medio de NEH, la destrucción aleatoria (<i>random</i>) y la guiada (<i>idle-rcl</i> , $\alpha = 0,30$) por <i>benchmark</i> . Derecha: diferencia de RPD entre la guiada y la aleatoria por <i>benchmark</i> (los valores negativos indican que la guiada mejora); el signo se invierte con la dificultad de las instancias.	39
7.1	Planificación temporal del proyecto (diagrama de Gantt). Las barras hasta julio reflejan el trabajo realizado; las posteriores, la planificación prevista. Los hitos marcan la solicitud de defensa (13–14 jul) y la entrega de la memoria (4–7 sep).	50

ÍNDICE DE TABLAS

2.1	Síntesis de la literatura revisada sobre el PFSP y el <i>Iterated Greedy</i>	13
3.1	Entorno de <i>hardware</i> y <i>software</i> de la experimentación (portátil y servidor). . .	23
4.1	Resumen de los experimentos: objetivo, algoritmo, datos y métrica. UB (<i>upper bound</i>): mejor <i>makespan</i> publicado para la instancia.	25
4.2	RPD del NEH sobre Taillard (EXP-001), por grupo $n \times m$	26
4.3	Calibración del presupuesto del IG (EXP-003): RPD medio y mejora marginal.	28
4.4	Aceleración (EXP-005): tiempo por ejecución a presupuesto 500 y <i>speedup</i> . Tiempos medidos en el equipo portátil descrito en el entorno de experimentación; EXP-003 y EXP-005 se ejecutaron en la misma máquina, de modo que el cociente es homogéneo.	30
4.5	Calibración del tamaño de la RCL (EXP-006): RPD medio por α	33
4.6	Calidad de las tres vías de destrucción (EXP-007): RPD medio por <i>benchmark</i> y rango medio de Friedman, sobre 600 instancias y 5 semillas. . . .	35
4.7	Destrucción guiada frente a la aleatoria, instancia a instancia (EXP-007). Un Δ RPD negativo indica que la guiada mejora.	36
6.1	Componentes de software de terceros y sus licencias.	47
7.1	Costes de personal.	51
7.2	Costes de hardware (amortización lineal a cinco años).	52
7.3	Costes de software.	52
7.4	Resumen de costes y total del proyecto.	53

LISTA DE ACRÓNIMOS

ARPD	<i>Average Relative Percentage Deviation</i> (desviación porcentual relativa media)
CD	<i>Critical Difference</i> (diferencia crítica, prueba de Nemenyi)
CLI	<i>Command-Line Interface</i> (interfaz de línea de órdenes)
CPLEX	IBM ILOG CPLEX Optimizer (<i>solver</i> comercial)
ECTS	<i>European Credit Transfer and Accumulation System</i>
ERP	<i>Enterprise Resource Planning</i>
GRASP	<i>Greedy Randomized Adaptive Search Procedure</i>
IA	Inteligencia artificial
IG	<i>Iterated Greedy</i>
JIT	<i>Just-In-Time</i> (compilación)
LB	<i>Lower Bound</i> (cota inferior)
MES	<i>Manufacturing Execution System</i>
MVP	<i>Minimum Viable Product</i> (producto mínimo viable)
NEH	Heurística de Nawaz, Enscore y Ham
NP	<i>Nondeterministic Polynomial time</i>
ODS	Objetivos de Desarrollo Sostenible
PFSP	<i>Permutation Flow-Shop Scheduling Problem</i>
PYME	Pequeña y Mediana Empresa
RCL	<i>Restricted Candidate List</i> (lista restringida de candidatos)
RGPD	Reglamento General de Protección de Datos
RPD	<i>Relative Percentage Deviation</i> (desviación porcentual relativa)
SA	<i>Simulated Annealing</i> (recocido simulado)
TFG	Trabajo de Fin de Grado
UB	<i>Upper Bound</i> (cota superior; mejor <i>makespan</i> publicado)
UC3M	Universidad Carlos III de Madrid
VRF	<i>Benchmark</i> de Vallada, Ruiz y Framinan

1. INTRODUCCIÓN

Este capítulo introduce el trabajo y delimita su alcance. En primer lugar se plantea el problema que se aborda (la programación de tareas en un *flow-shop* de permutación) y se justifica su relevancia. A continuación se expone la motivación, enraizada en las necesidades de las pequeñas y medianas empresas, y se enumeran los objetivos, general y específicos, que guían el desarrollo. El capítulo cierra describiendo la estructura del resto del documento.

1.1. Planteamiento del problema

El problema que aborda este trabajo es el *Permutation Flow-Shop Scheduling Problem* (PFSP): dado un conjunto de n trabajos que deben procesarse en m máquinas dispuestas en serie, donde todos los trabajos visitan las máquinas en el mismo orden y mantienen la misma secuencia en todas ellas, se busca el orden de los trabajos que minimiza el *makespan* (C_{max}): el instante en que el último trabajo termina en la última máquina. Es un problema de optimización combinatoria NP-duro [1], lo que hace inviables los métodos exactos a escala industrial. La Figura 1.1 ilustra el problema mediante un diagrama de Gantt: los trabajos recorren las máquinas en serie y el objetivo es ordenarlos para que el último termine cuanto antes. La formalización matemática completa (variables, restricciones y recurrencia del *makespan*) se desarrolla en el Capítulo 2 (*Estado del arte*).

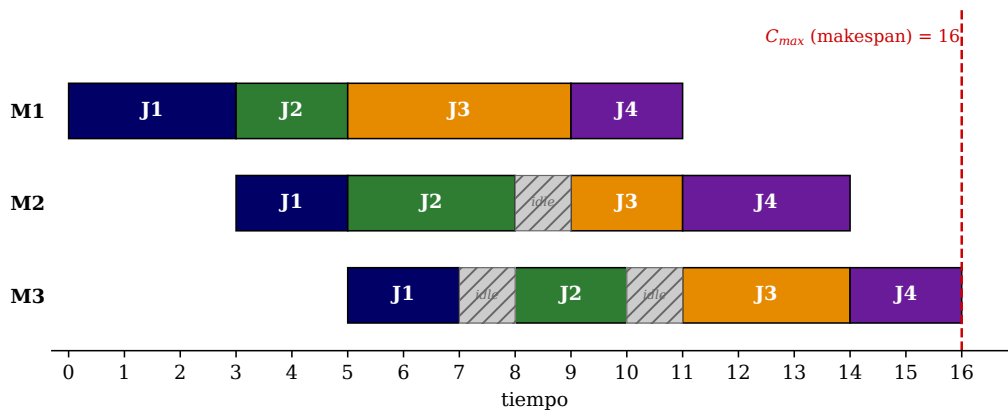


Fig. 1.1. Ejemplo de *flow-shop* de permutación con tres máquinas y cuatro trabajos. El *makespan* (C_{max}) es el instante en que el último trabajo termina en la última máquina; en gris, los tiempos muertos (*idle*).

1.2. Motivación y antecedentes

La programación de tareas en entornos de fabricación es un problema operativo recurrente: dado un conjunto de pedidos que deben recorrer una línea de máquinas en serie, el orden en que se introducen condiciona el tiempo total de producción. Este trabajo se centra en su formulación canónica, el PFSP, con el objetivo de minimizar el *makespan*.

Dado que el PFSP es NP-duro [1], los métodos exactos no resuelven en tiempos admisibles las instancias de tamaño industrial, y las herramientas que los implementan de forma competitiva (los *solvers* comerciales como Gurobi [2] o CPLEX [3]) suponen un coste de licencia y un conocimiento especializado fuera del alcance de muchas organizaciones. Esta barrera afecta de manera particular a las pequeñas y medianas empresas (PYMES), que rara vez disponen de infraestructura de cómputo de alto rendimiento ni de personal dedicado a la optimización.

De esta tensión surge la motivación del trabajo: ofrecer una solución de optimización de bajo coste computacional y *plug-and-play*, ejecutable en equipos convencionales y sin dependencia de licencias comerciales, que acerque la mejora de la planificación de la producción al tejido industrial a pequeña escala. Como beneficio intrínseco y narrativo (no como métrica modelada), reducir el *makespan* y los tiempos muertos (*idle times*) disminuye el tiempo que la maquinaria permanece encendida en espera, en línea con las prácticas de *Green Scheduling*.

El trabajo se apoya en dos técnicas consolidadas del área: la heurística constructiva de Nawaz, Ensore y Ham (NEH) [4], como línea base, y la metaheurística *Iterated Greedy* (IG) [5], reconocida por combinar eficacia y simplicidad. Sobre el IG se propone la aportación central: un operador de destrucción inteligente que, en lugar de extraer trabajos al azar de toda la secuencia, restringe ese sorteo a los que más tiempo muerto generan. La aleatoriedad no desaparece, se acota; como muestran los resultados, conservarla es precisamente lo que hace aprovechable la guía. La propuesta se valida sobre el conjunto de instancias de alta dureza de Vallada, Ruiz y Framinan (VRF) [6]. El estado del arte que sustenta estas decisiones se desarrolla en el Capítulo 2 (*Estado del arte*).

1.3. Objetivos

El objetivo general del trabajo es diseñar, implementar y validar un algoritmo basado en *Iterated Greedy*, de bajo coste computacional, para la resolución del PFSP con minimización del *makespan*, orientado a su aplicación en PYMES. La función objetivo que se optimiza es únicamente el *makespan*: el coste de cómputo no forma parte de ella, sino que se mide y se reporta como segundo eje de evaluación, puesto que de él depende que la solución resulte viable en un equipo convencional. Este objetivo se concreta en los siguientes objetivos específicos:

1. *Definición y modelado*. Formalizar matemáticamente el PFSP y contextualizar su

impacto operativo y económico en el tejido de las PYMEs.

2. *Implementación base (baseline)*. Desarrollar la heurística constructiva NEH y evaluarla sobre las instancias clásicas de Taillard como prueba de concepto.
3. *Desarrollo metaheurístico*. Implementar el *Iterated Greedy*, integrar el operador de destrucción inteligente y calibrar sus parámetros de aceptación.
4. *Estudio computacional riguroso*. Evaluar el algoritmo sobre los dos *benchmarks* de referencia (las instancias clásicas de Taillard y el conjunto de alta dureza VRF) en los dos ejes que determinan su utilidad práctica: la calidad de las soluciones, medida con la *Relative Percentage Deviation* (RPD) y contrastada mediante pruebas estadísticas no paramétricas, y el coste de cómputo necesario para alcanzarla.
5. *Propuesta de despliegue (producto mínimo viable)*. Proporcionar un producto mínimo viable (MVP) que permita reproducir y poner en marcha la solución: un repositorio con las dependencias, las instrucciones de instalación, compilación y ejecución, de modo que una PYME (o cualquiera que desee continuar el trabajo) pueda clonarlo y utilizarlo en un equipo convencional, sin necesidad de una arquitectura de despliegue industrial completa.

1.4. Estructura del documento

El resto del documento se organiza como sigue. El Capítulo 2 (*Estado del arte*) formaliza el PFSP y su intratabilidad, define el cálculo del *makespan* y la descomposición del tiempo muerto, y revisa la evolución de las técnicas de resolución hasta situar la oportunidad de mejora que fundamenta la aportación. El Capítulo 3 (*Diseño y metodología*) describe la arquitectura del software, los algoritmos implementados —la heurística NEH y el *Iterated Greedy*, con el operador de destrucción inteligente propuesto— y el diseño experimental (*benchmarks*, métrica y garantías de reproducibilidad). El Capítulo 4 (*Resultados experimentales*) presenta y analiza las campañas: la validación de la línea base, la calibración del algoritmo y la comparación de la aportación frente a la destrucción aleatoria. El Capítulo 5 (*Conclusiones y trabajo futuro*) recapitula el grado de cumplimiento de los objetivos, expone las limitaciones y apunta las líneas de continuación. Por último, el Capítulo 6 (*Marco regulador*) y el Capítulo 7 (*Entorno socio-económico y presupuesto*) abordan, respectivamente, la legislación y las licencias aplicables y el impacto socioeconómico, la planificación y el presupuesto del proyecto. El documento se cierra con la bibliografía y dos anexos: la guía de reproducción del producto mínimo viable y la declaración de uso de inteligencia artificial generativa.

2. ESTADO DEL ARTE

Este capítulo establece el marco teórico y revisa el cuerpo de conocimiento sobre el *Permutation Flow-Shop Scheduling Problem* (PFSP) y, en particular, sobre su resolución mediante el algoritmo *Iterated Greedy* (IG). La motivación del trabajo y sus objetivos se presentan en el Capítulo 1; este capítulo se circunscribe a los fundamentos y a la revisión de la literatura. Su objetivo es doble: situar el problema en su contexto teórico —su formalización, su intratabilidad computacional y la consiguiente necesidad de métodos aproximados— y trazar la evolución de las técnicas algorítmicas hasta identificar la oportunidad de mejora que fundamenta la aportación de este trabajo.

La exposición se organiza de lo general a lo específico. Primero se formaliza el problema, se justifica (a partir de resultados clásicos de complejidad) por qué los métodos exactos resultan inviables a escala industrial, y se introducen las dos nociones cuantitativas que vertebran el resto del trabajo: el cálculo del *makespan* y la descomposición del tiempo muerto (*idle time*). A continuación se ofrece una panorámica de las familias de métodos de resolución, que sirve para situar las dos técnicas sobre las que se asienta el proyecto: la heurística constructiva NEH y la metaheurística *Iterated Greedy*. Seguidamente se examina la evolución del IG durante la última década y, dentro de ella, el papel del operador de destrucción, cuya formulación habitual abre el hueco que este trabajo aborda. Por último, se discute el marco de evaluación experimental (los conjuntos de instancias de referencia y la métrica de comparación), se resume la literatura en una tabla comparativa y se sintetiza el posicionamiento del trabajo respecto al estado del arte.

Las fuentes se han seleccionado priorizando los artículos seminales de cada técnica y las revisiones y propuestas más citadas en revistas de referencia del área de investigación de operaciones, complementadas con trabajos recientes que ilustran las líneas de evolución activas.

2.1. El problema del flow-shop de permutación y su intratabilidad

El PFSP considera un conjunto de n trabajos que deben procesarse en m máquinas dispuestas en serie. Cada trabajo visita todas las máquinas en el mismo orden (de la máquina 1 a la máquina m) y, una vez iniciado su procesamiento en una máquina, no puede interrumpirse. Se denota por $p_{i,j} \geq 0$ el tiempo de procesamiento (determinista) del trabajo j en la máquina i . Una solución es una permutación $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ de los n trabajos, donde π_k indica el trabajo situado en la posición k de la secuencia.

La condición que define la variante de permutación (la *restricción de permutación*) es que el orden de los trabajos sea idéntico en todas las máquinas. Esta restricción es la que reduce drásticamente el tamaño del problema: mientras que un *flow-shop* general admite

una secuencia distinta en cada máquina, con $(n!)^m$ combinaciones posibles, la restricción de permutación limita el espacio de búsqueda a las $n!$ permutaciones de los trabajos.

El objetivo considerado en este trabajo es la minimización del *makespan* (C_{max}), esto es, el instante en que el último trabajo finaliza su procesamiento en la última máquina. En la notación de tres campos habitual en *scheduling*, el problema se expresa como $F_m \mid prmu \mid C_{max}$, donde F_m designa el *flow-shop* de m máquinas, *prmu* la restricción de permutación y C_{max} el criterio a optimizar [7]. La recurrencia que permite calcular el *makespan* de una permutación se detalla en la Sección 2.2.

La dificultad del problema depende críticamente del número de máquinas. Para $m = 2$, la regla de Johnson obtiene la secuencia óptima en tiempo polinómico $O(n \log n)$, resultado fundacional del *scheduling* de *flow-shop* [8]. Sin embargo, este resultado no se generaliza: Garey, Johnson y Sethi demostraron que la minimización del *makespan* en un *flow-shop* de m máquinas es NP-completa para $m \geq 3$ [1]. En consecuencia, el PFSP es NP-duro en sentido fuerte, y el tiempo de cómputo de cualquier método exacto — programación lineal entera mixta, ramificación y acotación— crece de forma prohibitiva con el tamaño de la instancia.

Esta intratabilidad tiene una implicación directa para el contexto industrial que motiva el trabajo. A escala real, donde el número de trabajos y máquinas puede ser elevado, los métodos exactos no producen soluciones en tiempos admisibles, y las herramientas que los implementan de forma competitiva (los *solvers* comerciales como Gurobi [2] o CPLEX [3]) conllevan además licencias costosas y conocimiento especializado. Por ello, la literatura ha dirigido el esfuerzo hacia métodos aproximados (heurísticas y metaheurísticas) capaces de obtener soluciones de alta calidad con un coste computacional moderado, que son los que se revisan más adelante.

2.2. Cálculo del *makespan*

Dada una permutación π , el *makespan* se obtiene mediante una recurrencia sobre los tiempos de finalización. Sea C la matriz de tiempos de finalización, de dimensión $m \times n$ (una fila por máquina y una columna por posición de la secuencia), y sea $C_{i,k}$ su elemento genérico: el instante en que el trabajo situado en la posición k de la secuencia termina su procesamiento en la máquina i . El cálculo parte del primer trabajo en la primera máquina y se propaga por filas (máquinas) y columnas (posiciones) atendiendo a dos dependencias: un trabajo no puede empezar en la máquina i hasta haber terminado en la máquina $i - 1$, ni hasta que la máquina i haya liberado al trabajo anterior de la secuencia.

Los casos frontera fijan el origen de la recurrencia. El primer trabajo en la primera máquina arranca en el instante cero, de modo que

$$C_{1,1} = p_{1,\pi_1}. \quad (2.1)$$

El primer trabajo en las máquinas restantes solo depende de su finalización en la máquina anterior,

$$C_{i,1} = C_{i-1,1} + p_{i,\pi_1}, \quad i = 2, \dots, m, \quad (2.2)$$

y los trabajos restantes en la primera máquina se encadenan unos tras otros,

$$C_{1,k} = C_{1,k-1} + p_{1,\pi_k}, \quad k = 2, \dots, n. \quad (2.3)$$

En el caso general, un trabajo no puede comenzar en la máquina i hasta que se satisfagan simultáneamente las dos dependencias, por lo que su inicio queda determinado por la mayor de ambas restricciones:

$$C_{i,k} = \max(C_{i-1,k}, C_{i,k-1}) + p_{i,\pi_k}, \quad i = 2, \dots, m; \quad k = 2, \dots, n. \quad (2.4)$$

El *makespan* de la permutación es el tiempo de finalización del último trabajo en la última máquina:

$$C_{\max}(\pi) = C_{m,n}. \quad (2.5)$$

La evaluación de las ecuaciones (2.1)–(2.5) tiene un coste de $O(nm)$ y constituye la operación más repetida en los algoritmos del trabajo. Sobre esta recurrencia se apoyan dos mejoras de naturaleza distinta, ambas empleadas en este trabajo y descritas en el Capítulo 3 (*Diseño y metodología*). La primera aprovecha la estructura regular del llenado de la matriz para calcularla con operaciones vectorizadas en lugar de un doble bucle: no altera el coste, que sigue siendo $O(nm)$, pero reduce el factor constante. La segunda es la evaluación incremental del cálculo de inserción [9], que sí rebaja el coste asintótico de evaluar todas las posiciones de inserción y es la que hace viables NEH y el IG sobre instancias grandes.

2.3. El tiempo muerto y su descomposición

El *idle time* (tiempo muerto) es el tiempo durante el cual una máquina está disponible pero no procesa ningún trabajo. En un diagrama de Gantt del *flow-shop*, se corresponde con los huecos vacíos entre operaciones consecutivas en una misma máquina. Reducir el tiempo muerto está directamente ligado a reducir el *makespan*, ya que ambos miden ineficiencia en el uso de la línea de producción. Conviene precisar que, en este trabajo, el tiempo muerto se trata exclusivamente como magnitud temporal: la mejora de eficiencia que conlleva su reducción se interpreta de forma narrativa, sin modelar el consumo energético.

La literatura de heurísticas de *flow-shop* descompone el tiempo muerto en dos contribuciones de naturaleza distinta [10]:

- *Front delay*: la máquina permanece inactiva a la espera de que el trabajo termine en la máquina anterior, es decir, el trabajo todavía no ha llegado a la máquina actual.
- *Back delay*: la máquina retiene un trabajo ya completado porque la máquina siguiente sigue ocupada y no puede recibirlo; este efecto se agrava en la variante con bloqueo (*blocking flow-shop*).

El tiempo muerto tiene un complemento natural en la noción de *ruta crítica*, que se emplea en varios puntos de este capítulo y conviene fijar aquí. La recurrencia (2.4) obtiene cada $C_{i,k}$ sumando p_{i,π_k} al mayor de dos términos, de modo que recorrer la matriz hacia atrás desde $C_{m,n}$ y escoger en cada paso el término que alcanzó el máximo reconstruye la cadena de operaciones que ha fijado el *makespan*. Esa cadena es la ruta crítica: una sucesión de operaciones que empieza en la primera máquina, acaba en la última y avanza en cada paso o bien a la posición siguiente de la misma máquina, o bien a la máquina siguiente del mismo trabajo. Dos propiedades explican su utilidad. La primera es que los tiempos de procesamiento de sus operaciones suman exactamente $C_{max}(\pi)$. La segunda es que la ruta no contiene holgura alguna, pues cada una de sus operaciones comienza en el instante en que termina la anterior; retrasar cualquiera de ellas prolonga el *makespan* en la misma medida. Cuando la ecuación (2.4) produce empates, la ruta crítica no es única. Los tramos maximales que recorre sobre una misma máquina se conocen como *bloques críticos* y constituyen la estructura que explota la búsqueda tabú de Nowicki y Smutnicki [11], comentada en la Sección 2.4.

Ambas nociones proporcionan el vocabulario estructural que permite distinguir qué trabajos generan inactividad y dónde, y su interés para la aportación del trabajo reside en cómo se combinan. Los trabajos que la ruta crítica atraviesa condicionan directamente el valor de C_{max} , mientras que los que quedan fuera disponen de holgura y pueden reubicarse sin alterarlo. Un operador de destrucción inteligente puede aprovechar esta asimetría para extraer preferentemente los trabajos responsables del mayor *front delay* en las máquinas críticas, preservando los bloques de trabajos ya bien empaquetados. El diseño concreto de este operador y su integración en el *Iterated Greedy* se desarrollan en el Capítulo 3.

2.4. Panorámica de los métodos de resolución

Los métodos propuestos para el PFSP pueden ordenarse en tres grandes familias, de mayor a menor garantía de optimalidad y, de forma inversa, de menor a mayor escalabilidad. La primera la forman los *métodos exactos* (programación lineal entera mixta y, sobre todo, esquemas de ramificación y acotación), que garantizan la solución óptima pero cuyo coste crece de forma prohibitiva con el tamaño de la instancia. Trabajos recientes han mejorado notablemente su eficiencia, como el algoritmo de ramificación y acotación de

Gmys et al. [12], pero su aplicabilidad sigue restringida a instancias pequeñas o medianas, lejos de los tamaños industriales.

La segunda familia la componen las *heurísticas constructivas*, que edifican una única solución mediante reglas deterministas sin explorar el espacio de búsqueda. Son rápidas y de implementación sencilla; su exponente es el algoritmo NEH [4], que se detalla en la sección siguiente. Su limitación es que producen una sola solución, sin mecanismo para mejorarla más allá de la regla de construcción.

La tercera familia, la de las *metaheurísticas*, explora iterativamente el espacio de soluciones combinando intensificación y diversificación. Las revisiones del área documentan una proliferación de propuestas (desde búsquedas locales y métodos de trayectoria hasta algoritmos poblacionales bioinspirados) y advierten de una tendencia a la complejidad creciente que no siempre se traduce en mejores resultados [13], [14]. En este contexto, el *Iterated Greedy* se ha consolidado como una metaheurística de referencia precisamente por contradecir esa tendencia: obtiene resultados de primer nivel con un esquema simple y pocos parámetros [5]. Este trabajo se inscribe en esta tercera familia y, dentro de ella, en la línea del IG.

Dentro de las metaheurísticas, conviene nombrar las familias que han competido con el IG en el PFSP, pues su comparación justifica la elección del IG. Entre los métodos de trayectoria, el *Simulated Annealing* de Osman y Potts introdujo la aceptación probabilística de soluciones peores [15], y la búsqueda tabú de Nowicki y Smutnicki (que restringe el vecindario a los bloques de la ruta crítica) se considera la referencia veloz de su clase [11]. Entre los métodos poblacionales y bioinspirados, el algoritmo genético de Reeves fue la primera adaptación formal de la lógica evolutiva al problema [16], y las colonias de hormigas de Rajendran y Ziegler ejemplifican el enfoque de inteligencia de enjambre [17]. La evidencia comparativa, sin embargo, es consistente: revisiones y estudios recientes muestran que el IG, más simple y parsimonioso, iguala o supera a estas metaheurísticas más complejas [13], [14], [18], lo que sustenta su adopción como algoritmo base de este trabajo.

2.5. Heurísticas constructivas: el algoritmo NEH

Entre las heurísticas constructivas para el PFSP, el algoritmo propuesto por Nawaz, Enscore y Ham (conocido por sus iniciales como NEH) es la referencia consolidada y sigue siendo, varias décadas después de su publicación, el punto de partida y el patrón de comparación habitual [4]. Su funcionamiento se basa en el principio de que los trabajos con mayor tiempo total de procesamiento deben situarse en las primeras posiciones de la secuencia. El procedimiento consta de dos fases: primero, los trabajos se ordenan de forma decreciente según la suma de sus tiempos de procesamiento en todas las máquinas; segundo, se construye la secuencia de manera incremental, insertando cada trabajo, uno a uno, en la posición de la secuencia parcial que minimiza el *makespan* parcial.

La calidad de las soluciones que produce NEH, unida a su simplicidad, explica su papel central en el área. No obstante, su formulación directa requiere evaluar el *makespan* en todas las posiciones de inserción posibles, lo que resulta costoso. Taillard introdujo un conjunto de aceleraciones basadas en el cálculo incremental de matrices de tiempos de finalización «hacia adelante» y «hacia atrás», que permiten evaluar todas las inserciones de un trabajo con un coste global del orden de $O(n^2m)$ [9]. Estas aceleraciones son hoy estándar en cualquier implementación eficiente de NEH y, por extensión, de los algoritmos que lo emplean como subrutina de construcción.

El interés de NEH para este trabajo es doble. Por un lado, constituye la línea base (*baseline*) constructiva frente a la que se contrastan los métodos más elaborados. Por otro, su mecánica de inserción es precisamente la que el *Iterated Greedy* reutiliza en su fase de reconstrucción, como se expone en la sección siguiente.

2.6. Iterated Greedy: el algoritmo de referencia

El *Iterated Greedy* (IG), formulado por Ruiz y Stützle para el PFSP, es la metaheurística que se ha consolidado como referencia para este problema [5]. Su esquema es deliberadamente simple. A partir de una solución inicial (construida habitualmente con NEH), el algoritmo itera tres fases hasta agotar el tiempo de cómputo: una fase de destrucción, que extrae un número fijo d de trabajos de la secuencia; una fase de construcción, que reinserta esos trabajos uno a uno mediante la mecánica codiciosa de NEH; y un criterio de aceptación que decide si la nueva solución reemplaza a la actual. Este criterio, de tipo *Simulated Annealing*, admite empeoramientos con una probabilidad que depende de un parámetro de temperatura (alta al principio y decreciente a lo largo de la búsqueda), lo que permite escapar de óptimos locales sin perder el rumbo. Opcionalmente, se aplica una búsqueda local entre fases.

La relevancia del IG no reside en su sofisticación, sino en lo contrario. Ruiz y Stützle demostraron que este esquema, gobernado por apenas dos parámetros (el tamaño de destrucción d y la temperatura), supera estadísticamente a métodos híbridos considerablemente más complejos [5]. Esta combinación de eficacia y parsimonia paramétrica explica que el IG se haya convertido en el patrón frente al que se contrastan las propuestas posteriores, y es también la razón por la que se adopta como algoritmo base de este trabajo: ofrece un punto de partida competitivo y de bajo coste de configuración, coherente con el enfoque dirigido a la pequeña y mediana empresa. El detalle de cada fase, los valores adoptados para sus parámetros y el pseudocódigo del bucle se recogen en la Sección 3.3; a los efectos de esta revisión basta retener el esquema de tres fases, que es el que comparten las extensiones examinadas a continuación.

2.7. Evolución del Iterated Greedy (2014–2024)

Desde su formulación, el IG ha sido objeto de numerosas extensiones que pueden agruparse en tres líneas. La primera busca mejorar la calidad de la solución intermedia: Dubois-Lacoste, Pagnozzi y Stützle identifican la optimización de la secuencia parcial como un factor crítico y aplican una búsqueda local sobre la subsecuencia residual antes de la reinserción, con el argumento de que reconstruir sobre un bloque parcial subóptimo arrastra ineficiencias hasta la solución final [19].

La segunda línea demuestra la plasticidad del esquema ante variantes del problema. Pan y Ruiz adaptan el IG al *flow-shop* de permutación *no-idle* (donde la inactividad de las máquinas resulta costosa, como en la industria química o metalúrgica), rediseñando la evaluación de los tiempos de finalización sin alterar el bucle metaheurístico [20]. Ruiz, Pan y Naderi lo extienden al PFSP distribuido, con múltiples fábricas idénticas, y muestran que un IG simple con buena inicialización y búsqueda local supera a heurísticas bioinspiradas más complejas [18]. Ambos trabajos refuerzan una misma tesis: el diseño heurístico riguroso prima sobre las metáforas biológicas.

La tercera línea introduce perturbación guiada y aprendizaje. Fernández-Viagas y Framinan observan que, durante el procedimiento NEH, numerosas inserciones empatan en *makespan* parcial, y que romper esos empates atendiendo al tiempo muerto residual (en lugar de por el primer índice) mejora notablemente la convergencia, al favorecer secuencias más compactas [21]. En la frontera actual, Karimi-Mamaghan et al. integran *Q-learning* en el IG: el algoritmo dispone de un portafolio de operadores de destrucción y aprende, según el estado de la búsqueda, cuál aplicar, superando a las estrategias de perturbación estáticas [22]. En esa misma dirección, Ying y Lin combinan aprendizaje por refuerzo con IG para el PFSP de ensamblaje distribuido [23]. Estas propuestas delimitan el estado del arte de la perturbación adaptativa, si bien su complejidad (aprendizaje por refuerzo completo) se aleja del objetivo de bajo coste de este trabajo.

2.8. El operador de destrucción: aleatorio frente a guiado

El repaso anterior permite situar el punto sobre el que incide este trabajo. En la formulación clásica del IG, la fase de destrucción extrae los d trabajos al azar [5]. Esta elección es sencilla y eficaz, pero ciega al estado de la solución: la extracción aleatoria rompe con frecuencia bloques de trabajos que ya estaban bien empaquetados, obligando a la fase de reconstrucción a reparar un daño que no era necesario producir.

La literatura de perturbación guiada sugiere que esta limitación puede mitigarse inyectando conocimiento del dominio en el operador. Las reglas de desempate basadas en tiempo muerto [21] y la selección adaptativa de operadores de destrucción (incluidos operadores guiados por la ruta crítica o por el perfil de holgura) [22] evidencian que orientar la perturbación con información estructural de la solución mejora la convergencia res-

pecto a la aleatoriedad pura. El concepto que articula esta información es el *idle time* (tiempo muerto), cuya descomposición en *front delay* y *back delay* (formalizada en la Sección 2.3) proporciona el vocabulario estructural con el que un operador de destrucción puede distinguir qué trabajos generan inactividad y cuáles no.

Esta línea de guiado por tiempo muerto ha cobrado fuerza recientemente. En el plano constructivo, Liu, Jin y Price proponen una regla de desempate de NEH que minimiza conjuntamente *makespan* y tiempo muerto de máquina, mostrando que comprimir el *idle time* durante la inserción mejora la calidad de las soluciones [24]. Dentro ya del bucle del IG, Ji et al. integran la inserción guiada por la posición de los tiempos de inactividad mediante grupos de penalización [25], y Li et al. emplean grafos disyuntivos para restringir la destrucción y la búsqueda a las operaciones de la ruta crítica [26]. Aunque varias de estas propuestas se formulan sobre variantes (p. ej. *hybrid flow shop*), comparten el mecanismo que interesa a este trabajo: dirigir la perturbación con información de *idle time* y ruta crítica en lugar de hacerlo al azar.

Aquí se sitúa la oportunidad que aborda el trabajo: sustituir la destrucción aleatoria uniforme por un operador de destrucción inteligente que concentre la extracción en los trabajos responsables del mayor tiempo muerto en las máquinas críticas, preservando los bloques eficientes, y que lo haga sin renunciar al componente aleatorio del que el método obtiene su diversificación. El diseño concreto del operador se desarrolla en el Capítulo 3.

2.9. Marco de evaluación: Taillard frente a VRF

La comparación de algoritmos para el PFSP se apoya en conjuntos de instancias de referencia y en una métrica común. El primer estándar global fue el propuesto por Taillard, con 120 instancias pseudoaleatorias estratificadas que abarcan combinaciones de trabajos y máquinas entre 20×5 y 500×20 [27]. Durante décadas permitió contrastar heurísticas de forma homogénea mediante la *Relative Percentage Deviation* (RPD) frente a las mejores soluciones conocidas. Sin embargo, el progreso de los métodos aproximados ha provocado un «efecto techo»: las técnicas modernas alcanzan o rozan las mejores soluciones conocidas de Taillard, con lo que el conjunto pierde capacidad para discriminar entre algoritmos.

Para restaurar ese poder discriminante, Vallada, Ruiz y Framinan introdujeron el *benchmark* VRF: 480 instancias seleccionadas, tras una criba computacional masiva, por su dureza (medida como la separación entre cotas superiores e inferiores), con tamaños de hasta 800 trabajos y 60 máquinas [6]. Las revisiones del área respaldan tanto el diagnóstico como la solución: el *survey* fundacional de Ruiz y Maroto, anterior al IG, ya advertía de la fragilidad de los algoritmos densos al escalar [13], y la revisión moderna de Fernández-Viagas, Ruiz y Framinan (que emplea VRF a gran escala) concluye que muchas metaheurísticas recientes no superan al IG básico, y aboga por desarrollos parsimoniosos centrados en el problema [14]. En consecuencia, este trabajo valida la propuesta

sobre VRF con la métrica RPD y análisis estadístico, sin renunciar a Taillard como prueba de concepto inicial. Las instancias de VRF empleadas se obtuvieron de un espejo de descarga público asociado a la literatura de métodos exactos [12].

2.10. Síntesis comparativa y posicionamiento del trabajo

La Tabla 2.1 resume las contribuciones revisadas, ordenadas cronológicamente, con su aportación principal y su relación con este trabajo.

Referencia	Año	Aportación principal	Relación con el trabajo
Johnson [8]	1954	Regla óptima para el <i>flow-shop</i> de 2 máquinas	Resultado fundacional; caso tratable
Garey, Johnson y Sethi [1]	1976	NP-completitud del <i>flow-shop</i> para $m \geq 3$	Justifica el enfoque aproximado
Graham et al. [7]	1979	Notación de tres campos del <i>scheduling</i>	Notación del problema
King y Spachis [10]	1980	Heurísticas de <i>flow-shop</i> ; descomposición del <i>idle time</i> en <i>front</i> y <i>back delay</i>	Fundamento conceptual de la destrucción inteligente
Nawaz, Ensore y Ham [4]	1983	Heurística constructiva NEH	<i>Baseline</i> y núcleo de reconstrucción del IG
Osman y Potts [15]	1989	<i>Simulated annealing</i> para el PFSP	Rival (SA); aceptación que hereda el IG
Taillard [9]	1990	Aceleraciones del cálculo de inserción	Implementación eficiente de NEH/IG
Taillard [27]	1993	<i>Benchmark</i> clásico (120 instancias)	Prueba de concepto; «efecto techo»
Reeves [16]	1995	Algoritmo genético para <i>flow-shop</i>	Rival (GA); coste del enfoque poblacional
Nowicki y Smutnicki [11]	1996	Búsqueda tabú rápida (bloques de ruta crítica)	Rival (tabú); guiado por ruta crítica
Rajendran y Ziegler [17]	2004	Colonias de hormigas para el PFSP	Rival bioinspirado; alto coste
Ruiz y Maroto [13]	2005	<i>Survey</i> de heurísticas pre-IG	Contexto evolutivo
Ruiz y Stützle [5]	2007	Formulación del <i>Iterated Greedy</i>	Algoritmo base del trabajo
Fernández-Viagas y Framinan [21]	2014	Desempate por tiempo muerto en NEH	Antecedente de la destrucción inteligente
Pan y Ruiz [20]	2014	IG para el PFSP <i>no-idle</i>	Plasticidad del IG; vínculo con <i>idle time</i>
Vallada, Ruiz y Framinan [6]	2015	<i>Benchmark</i> VRF (480 instancias duras)	<i>Benchmark</i> de validación del trabajo
Liu, Jin y Price [24]	2016	NEH bicriterio: <i>makespan</i> + tiempo muerto	Antecedente directo de la destrucción guiada

Referencia	Año	Aportación principal	Relación con el trabajo
Fernández-Viagas, Ruiz y Framinan [14]	2017	<i>Survey</i> moderno con VRF	Aval del enfoque parsimonioso
Dubois-Lacoste, Pagnozzi y Stützle [19]	2017	IG con optimización de soluciones parciales	Inyección de inteligencia en el ciclo
Ruiz, Pan y Naderi [18]	2019	IG para el PFSP distribuido	Aval del enfoque parsimonioso
Gmys et al. [12]	2020	Ramificación y acotación eficiente	Método exacto; espejo de datos VRF
Ying y Lin [23]	2022	Aprendizaje por refuerzo + IG	Frontera RL; contraste de complejidad
Karimi-Mamaghan et al. [22]	2023	<i>Q-learning</i> para seleccionar operadores	Estado del arte de perturbación adaptativa
Ji et al. [25]	2024	IG con inserción guiada por tiempo muerto	Destrucción guiada por <i>idle time</i>
Li et al. [26]	2025	IG guiado por ruta crítica (grafo disyuntivo)	Destrucción guiada por ruta crítica

TABLA 2.1. SÍNTESIS DE LA LITERATURA REVISADA SOBRE EL PFSP Y EL *ITERATED GREEDY*.

La revisión precedente sostiene tres conclusiones que enmarcan el trabajo. Primera: el PFSP es un problema NP-duro cuya resolución a escala industrial exige métodos aproximados de bajo coste, sin dependencia de *solvers* comerciales. Segunda: entre esos métodos, el *Iterated Greedy* destaca por combinar eficacia y parsimonia, lo que lo hace idóneo como base de una solución dirigida a la pequeña y mediana empresa. Tercera: la fase de destrucción del IG clásico opera de forma aleatoria, y la evidencia sobre perturbación guiada indica que incorporar conocimiento del dominio —en particular, el tiempo muerto— puede mejorar la convergencia.

Sobre esta base, el trabajo propone un operador de destrucción inteligente que restrinja la extracción aleatoria del IG a los trabajos que más tiempo muerto generan, acotando el azar en lugar de suprimirlo, y evalúa su efecto sobre el *benchmark* VRF mediante RPD y análisis estadístico. La formulación del operador y su integración en el IG se desarrollan en los capítulos siguientes.

3. DISEÑO Y METODOLOGÍA

Este capítulo presenta el diseño algorítmico y la metodología experimental del trabajo. En primer lugar se describe la arquitectura del software desarrollado; a continuación se detallan los algoritmos implementados —la heurística constructiva NEH y la metaheurística *Iterated Greedy* (IG) clásica— y, por último, se expone el diseño experimental que guía la evaluación.

3.1. Arquitectura del software

La implementación se estructura en el paquete `pfsp`, desarrollado en Python con NumPy como única dependencia de cálculo. La arquitectura sigue un diseño modular que separa responsabilidades:

- **Núcleo algorítmico** (`pfsp.core`): cálculo del *makespan* (vectorizado con NumPy), heurística NEH, operadores del IG (destrucción, reconstrucción, búsqueda local) y mecánica de inserción compartida.
- **Entrada/salida** (`pfsp.io`): lectores de instancias en formato Taillard (`.fsp`) y VRF (`.txt`).
- **Resultados** (`pfsp.results`): registro individual por instancia (*record*), manifiestos de ejecución, agregación por grupo y cálculo del RPD.
- **Experimentación** (`pfsp.experiments`): *runner* de campaña reanudable con *checkpointing* por instancia.
- **Puntos de entrada por línea de órdenes** (CLI, *Command-Line Interface*): `pfsp-neh` (*baseline* NEH) y `pfsp-ig` (*Iterated Greedy*).

El enfoque persigue la viabilidad en un entorno PYME: sin licencias de *solvers* comerciales (Gurobi, CPLEX), sin compilador *Just-In-Time* (JIT; Numba se reserva solo si un perfilado lo justifica) y con resultados reproducibles mediante semillas fijas y datos crudos conservados íntegramente.

Conviene detallar en qué consiste la vectorización del cálculo del *makespan*, por ser la operación que más veces se ejecuta. La recurrencia (2.4) es secuencial en sus dos índices, de modo que no admite un cálculo simultáneo de toda la matriz. Sin embargo, la fila correspondiente a una máquina puede reescribirse en forma cerrada: llamando s a la suma acumulada de los tiempos de proceso de esa máquina y a a la fila de la máquina anterior, se cumple $C_{i,\cdot} = s + \max^{\text{acum}}(a - s_{-1})$, donde s_{-1} es la suma acumulada desplazada una posición y $\max^{\text{acum}}$ denota el máximo acumulado. Cada fila se obtiene así con unas

pocas operaciones vectorizadas sobre los n trabajos, y el único bucle explícito recorre las m máquinas, que son el orden de magnitud pequeño del problema. La complejidad no cambia, sigue siendo $O(nm)$: la mejora es de factor constante, por ejecutar el recorrido en las rutinas compiladas de NumPy en lugar de en un doble bucle interpretado. Esta misma rutina se reutiliza para construir las matrices de tiempos “hacia adelante” y “hacia atrás” sobre las que se apoya la evaluación acelerada de inserciones (Sección 3.2), de manera que ambas optimizaciones se combinan en lugar de excluirse.

3.2. Heurística constructiva NEH

La heurística NEH [4] es un método constructivo determinista que genera una permutación de buena calidad en tiempo polinómico. Se compone de dos etapas:

1. **Ordenación inicial.** Los n trabajos se ordenan por tiempo total de procesamiento decreciente:

$$T_j = \sum_{i=1}^m p_{i,j}, \quad j = 1, \dots, n. \quad (3.1)$$

En caso de empate se desempata por índice menor (criterio determinista).

2. **Construcción incremental.** Los trabajos se insertan uno a uno en la secuencia parcial. Para cada trabajo entrante, se evalúan todas las posiciones posibles y se elige la que produce el menor *makespan*; ante empate se selecciona la posición de índice más bajo.

La complejidad resultante es $O(n^2m)$, dado que en cada uno de los n pasos de inserción se evalúan hasta n posiciones, cada una con coste $O(m)$ gracias a la evaluación incremental acelerada de Taillard [9]. La Figura 3.1 ilustra el paso de inserción.

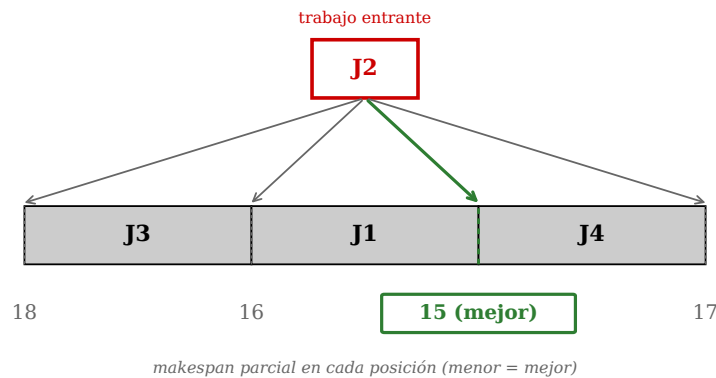


Fig. 3.1. Paso de inserción de NEH: el trabajo entrante se prueba en todas las posiciones de la secuencia parcial y se fija en la que minimiza el *makespan* parcial (valores ilustrativos).

3.3. Iterated Greedy (IG) clásico

El *Iterated Greedy* (IG) de Ruiz y Stützle [5] es una metaheurística de trayectoria que parte de una solución constructiva (NEH) y la mejora iterativamente mediante un ciclo de destrucción, reconstrucción codiciosa, búsqueda local y criterio de aceptación probabilístico. La Figura 3.2 resume el ciclo completo del algoritmo.

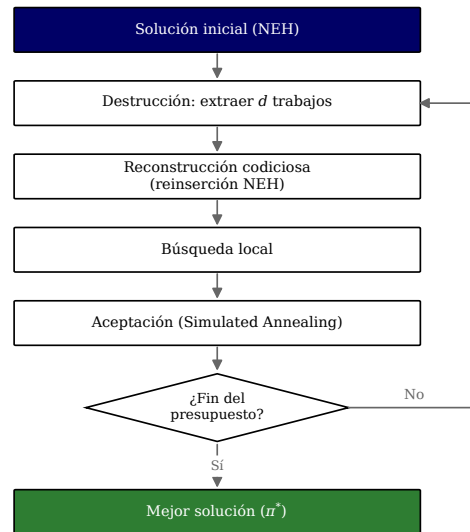


Fig. 3.2. Ciclo del *Iterated Greedy*: a partir de una solución inicial NEH, se itera destrucción, reconstrucción codiciosa, búsqueda local y aceptación hasta agotar el presupuesto.

3.3.1. Destrucción aleatoria

El operador de destrucción extrae d trabajos de la permutación actual, seleccionados uniformemente al azar (sin reemplazo), y los almacena en una lista de removidos preservando su orden relativo. El parámetro d controla la intensidad de la perturbación; el valor por defecto es $d = 4$, coherente con la recomendación de la literatura [5].

3.3.2. Construcción (reinserción NEH)

Los d trabajos extraídos se reinserían uno a uno en la secuencia parcial mediante la misma mecánica de `best_insertion` utilizada en la fase 2 del NEH: para cada trabajo, se evalúan todas las posiciones y se elige la de menor *makespan* (desempeño por índice más bajo). Este paso constituye una reconstrucción codiciosa local.

3.3.3. Búsqueda local de inserción

Tras la reconstrucción se aplica una búsqueda local de inserción: se recorren los trabajos en orden aleatorio (determinado por el generador semillado); para cada trabajo, se

extrae de su posición actual y se reinserta en la mejor posición disponible. Si la reinsertión mejora el *makespan*, se acepta y se reinicia el recorrido. El proceso se repite hasta completar un recorrido completo sin mejora.

3.3.4. Aceptación tipo Simulated Annealing

El criterio de aceptación decide si la nueva solución reemplaza a la solución incumbente (la mejor de la iteración). Se acepta siempre si mejora o iguala el *makespan* actual. En caso contrario, se acepta con probabilidad:

$$\text{Pr}[\text{aceptar}] = \exp\left(\frac{-(C_{\max}^{\text{nueva}} - C_{\max}^{\text{actual}})}{T}\right), \quad (3.2)$$

donde la temperatura T se calcula como:

$$T = T_p \cdot \frac{\sum_{i,j} P_{i,j}}{n \cdot m \cdot 10}, \quad (3.3)$$

con $T_p = 0,4$ por defecto. Este esquema de temperatura fija (no decreciente) permite escapar de óptimos locales a lo largo de toda la ejecución.

3.3.5. Criterio de parada

El criterio de parada es configurable: por número de evaluaciones, por número de iteraciones del bucle o por tiempo de CPU. El modo por defecto es el de evaluaciones, que garantiza reproducibilidad exacta al no depender de la velocidad del *hardware*. La implementación contabiliza una evaluación por iteración del bucle, de modo que ambos modos deterministas expresan el mismo presupuesto y las cifras son intercambiables; en adelante se reporta en iteraciones. El presupuesto se calibró experimentalmente en 500 iteraciones (véase la Sección 4.5).

3.3.6. Pseudocódigo del bucle IG

El Algoritmo 1 resume el flujo completo del *Iterated Greedy* implementado. Descrito en palabras, el procedimiento parte de una solución inicial construida con NEH y, opcionalmente, refinada con una búsqueda local, que se toma como mejor solución conocida. A partir de ahí repite un mismo ciclo hasta agotar el presupuesto de parada: primero destruye la solución actual extrayendo d trabajos; después los reconstruye, reinsertándolos uno a uno con la mecánica codiciosa de NEH; a continuación aplica una búsqueda local sobre la solución reconstruida; y finalmente un criterio de aceptación de tipo *Simulated Annealing* decide si la nueva solución sustituye a la actual —acepta siempre las que mejoran y, con una probabilidad que decrece con la temperatura, también algunas peores, lo que permite escapar de óptimos locales—. Cada vez que se mejora la mejor solución conocida, esta se actualiza. Al agotar el presupuesto, el algoritmo devuelve la mejor solución encontrada y su *makespan*.

Algoritmo 1 Iterated Greedy clásico (Ruiz y Stützle, 2005)

Entrada: Instancia (n, m, P) ; parámetros d, T_p ; criterio de parada

Salida: Mejor permutación π^* y su C_{max}

```
1:  $\pi \leftarrow \text{NEH}(n, m, P)$ 
2:  $\pi \leftarrow \text{BÚSQUEDALOCAL}(\pi)$ 
3:  $\pi^* \leftarrow \pi$ 
4: Calcular temperatura  $T$ 
5: while no se cumpla el criterio de parada do
6:    $R \leftarrow \text{DESTRUIR}(\pi, d)$                                 ▶ extraer  $d$  trabajos al azar
7:    $\pi' \leftarrow \text{RECONSTRUIR}(\pi \setminus R, R)$                   ▶ re inserción NEH
8:    $\pi' \leftarrow \text{BÚSQUEDALOCAL}(\pi')$ 
9:   if  $C_{max}(\pi') \leq C_{max}(\pi)$  then
10:     $\pi \leftarrow \pi'$ 
11:   else if  $\text{rand}() < \exp(-(C_{max}(\pi') - C_{max}(\pi))/T)$  then
12:     $\pi \leftarrow \pi'$ 
13:   end if
14:   if  $C_{max}(\pi) < C_{max}(\pi^*)$  then
15:     $\pi^* \leftarrow \pi$ 
16:   end if
17: end while
18: return  $\pi^*, C_{max}(\pi^*)$ 
```

3.4. Destrucción inteligente (aportación)

La aportación central del trabajo consiste en modificar la destrucción aleatoria del IG clásico para que el sorteo deje de recorrer la secuencia entera y quede restringido a los trabajos responsables del mayor *idle time*. La aleatoriedad no desaparece, se acota: el operador puntúa cada posición por el tiempo muerto que induce, retiene las de mayor puntuación en una lista de candidatos y extrae de ella d trabajos al azar. La hipótesis es que concentrar la eliminación en los cuellos de botella (en lugar de en trabajos arbitrarios) permite a la reconstrucción codiciosa dirigir su esfuerzo a la zona más prometedora del espacio de búsqueda, mejorando la convergencia con el mismo presupuesto de iteraciones.

La Figura 3.3 contrasta ambas estrategias sobre una misma secuencia. En el panel izquierdo, la destrucción aleatoria selecciona dos trabajos sin atender al estado de la solución, de modo que puede romper bloques ya bien empaquetados y obligar a la reconstrucción a rehacer un trabajo que no era necesario deshacer. En el derecho, la destrucción guiada localiza los huecos de inactividad de la máquina crítica (marcados en rojo) y limita el sorteo a los trabajos adyacentes a ellos, que son los que están generando ese tiempo muerto; los tramos que ya funcionan quedan fuera. La diferencia no está en *cuántos* trabajos se extraen, sino en el conjunto del que se sortean.

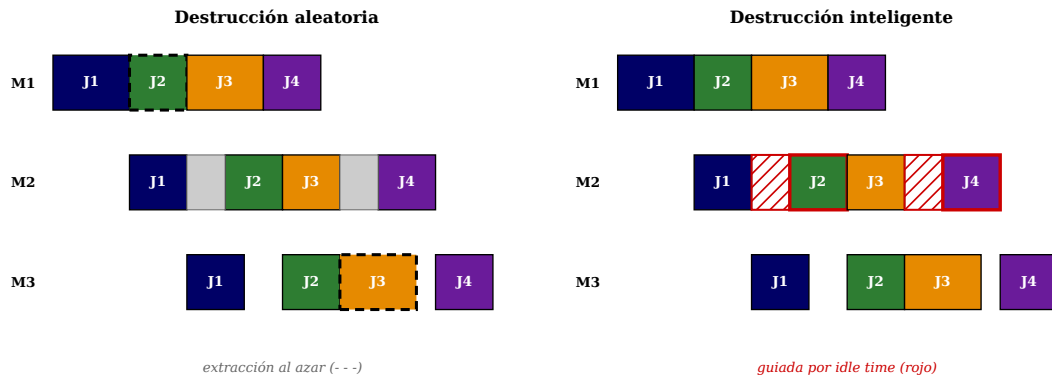


Fig. 3.3. Destrucción aleatoria (izquierda) frente a destrucción inteligente (derecha): esta última restringe el sorteo a los trabajos responsables del mayor tiempo muerto en las máquinas críticas, preservando los bloques eficientes.

El concepto se inspira en los resultados de Fernández-Viagas y Framinan [21], quienes demostraron que la inyección de conocimiento sobre el *idle time* en los operadores del NEH mejora la calidad de la solución constructiva.

3.4.1. Puntuación de tiempo muerto

La guía del operador se apoya en una puntuación que cuantifica el tiempo muerto que cada trabajo induce en la secuencia. Reutilizando la matriz de tiempos de finalización C definida por la recurrencia de la Sección 2.2 (donde $C_{i,k}$ es el instante en que el trabajo de la posición k termina en la máquina i), se define la puntuación de la posición k como el

front delay acumulado a lo largo de las máquinas:

$$\text{idle}(k) = \sum_{i=2}^m \text{máx}(0, C_{i-1,k} - C_{i,k-1}), \quad (3.4)$$

con la convención $C_{i,0} = 0$ para la primera posición. Cada término mide el tiempo que la máquina i permanece inactiva tras liberar el trabajo anterior ($C_{i,k-1}$) a la espera de que el trabajo de la posición k llegue desde la máquina precedente ($C_{i-1,k}$). Con esa convención, la primera posición acumula el tiempo muerto de *arranque* (*ramp-up*) $\text{idle}(1) = \sum_{i=1}^{m-1} C_{i,1}$ (la inactividad de las máquinas mientras el primer trabajo recorre la línea), que tiende a ser elevado. Un operador puramente voraz extraería por ello casi siempre esa primera posición, sesgando la búsqueda hacia una misma eliminación; el muestreo uniforme sobre la lista restringida de candidatos (RCL, *Restricted Candidate List*; Sección 3.4.2) mitiga ese sesgo, pues la posición de arranque pasa a seleccionarse solo con probabilidad $d/|\text{RCL}|$ en lugar de forma sistemática.

La puntuación es una función de la matriz C que ya se calcula para el *makespan*, por lo que obtenerla cuesta $O(nm)$ y no requiere una recurrencia adicional, en coherencia con el enfoque de bajo coste del trabajo.

3.4.2. Destrucción guiada por tiempo muerto

A partir de las puntuaciones se construye la RCL con las posiciones de mayor tiempo muerto. Su tamaño se fija como

$$|\text{RCL}| = \text{máx}(d, \lceil \alpha n \rceil), \quad (3.5)$$

donde d es el número de trabajos a extraer y $\alpha \in (0, 1]$ el parámetro que regula el grado de exploración (calibrado empíricamente en $\alpha = 0,30$; véase la Sección 4.6.1). Las posiciones se ordenan por tiempo muerto decreciente (con desempate determinista por índice de posición menor) y se toman las $|\text{RCL}|$ primeras. La cota inferior d garantiza que siempre haya al menos d candidatos que muestrear, incluso cuando $\alpha n < d$.

El operador de destrucción guiada por tiempo muerto (*idle-RCL*), aportación central del trabajo, procede en cada iteración del IG del siguiente modo: (i) calcula $\text{idle}(k)$ para toda posición mediante la Ecuación (3.4); (ii) construye la RCL según la Ecuación (3.5); y (iii) extrae d posiciones de la RCL muestreadas uniformemente y sin reemplazo con el generador aleatorio semillado. Los trabajos extraídos pasan a la lista de eliminados y los restantes conservan su orden relativo, exactamente en el mismo punto del bucle que ocupaba la destrucción aleatoria (Algoritmo 1).

La combinación de RCL y muestreo uniforme sigue el patrón GRASP (*Greedy Randomized Adaptive Search Procedure*): la RCL aporta la guía estructural (concentrar la eliminación en los trabajos que más tiempo muerto generan), mientras que el muestreo

uniforme añade la diversificación necesaria para no repetir siempre la misma eliminación, lo que evita el ciclado y el estancamiento de una selección puramente voraz.

3.4.3. Variante voraz y coste

Para aislar experimentalmente el efecto del muestreo se define una segunda variante, la destrucción voraz por tiempo muerto (*idle-greedy*), que extrae directamente las d posiciones de mayor tiempo muerto (desempate por índice menor), sin componente aleatoria. No es el operador propuesto, sino el término de comparación en la ablación (Sección 4.6.2): contrastar la variante voraz con la destrucción guiada mide cuánto aporta el muestreo sobre la RCL frente a la elección puramente voraz.

Ambos operadores solo sustituyen la fase de destrucción del Algoritmo 1 (el *Iterated Greedy* clásico); la reconstrucción, la búsqueda local y el criterio de aceptación no cambian, y el operador aleatorio sigue siendo el de por defecto. Su coste por iteración es $O(nm)$, el de la puntuación de tiempo muerto, del mismo orden que la evaluación del *mapspan*, de modo que la guía no añade sobrecoste apreciable. La selección adaptativa del operador mediante aprendizaje (por ejemplo, *Q-learning*) queda fuera del alcance por su coste de configuración, coherente con la orientación a la PYME, y se señala como trabajo futuro.

3.5. Diseño experimental

Esta sección define el marco en el que se evalúan los algoritmos. Se describen los conjuntos de instancias empleados (uno clásico como prueba de concepto y otro de alta dureza como validación), la métrica de calidad con la que se comparan los resultados, el número de réplicas y las garantías de reproducibilidad, y el funcionamiento del *runner* de campaña que orquesta las ejecuciones. El objetivo es que cualquier resultado presentado en el Capítulo 4 (*Resultados experimentales*) sea trazable y regenerable. En este trabajo se denomina campaña al conjunto completo de ejecuciones de un algoritmo sobre un *benchmark*, incluidas sus réplicas (una por cada semilla); cada campaña se identifica con un código EXP-NNN y genera su propio fichero de resultados crudos, del que se derivan las tablas y figuras.

3.5.1. Benchmarks

La evaluación se realiza sobre dos conjuntos de instancias complementarios:

- **Taillard (1993)** [27]: 120 instancias (12 grupos de 10 instancias, $n \in \{20, 50, 100, 200, 500\}$ y $m \in \{5, 10, 20\}$). Sirve como prueba de concepto, aunque presenta un “efecto techo” que limita su poder discriminante para métodos modernos.

- **VRF (2015)** [6]: 480 instancias (240 *Small* + 240 *Large*, hasta 800×60), filtradas por dureza. Restaura el poder de discriminación estadística y constituye el benchmark de validación principal.

3.5.2. Métrica de calidad

La métrica principal es la Desviación Porcentual Relativa (RPD):

$$RPD = 100 \cdot \frac{C_{max} - C_{max}^{ref}}{C_{max}^{ref}}, \quad (3.6)$$

donde C_{max}^{ref} es el *upper bound* (Taillard) o el mejor *makespan* conocido (VRF, del fichero *BestSolutionsAndBounds*).

Para algoritmos estocásticos se reporta el ARPD (*Average Relative Percentage Deviation*), es decir, el RPD medio sobre las réplicas.

3.5.3. Réplicas y reproducibilidad

Los algoritmos estocásticos (IG) se ejecutan con 5 semillas fijas por instancia (42, 123, 456, 789, 2025), lo que permite calcular la dispersión y garantizar la reproducibilidad exacta. El NEH, al ser determinista, requiere una única ejecución por instancia.

La reproducibilidad se asegura mediante:

- Semillas explícitas: todo componente que consume aleatoriedad recibe su generador de un único punto del paquete (`pfsp.repro.seeds.make_rng`), sin estado global implícito.
- Criterio de parada determinista (evaluaciones, equivalentes a iteraciones del bucle) como modo por defecto.
- Datos crudos completos preservados (el resumen se deriva, nunca se escribe a mano).

3.5.4. Runner de campaña

El *runner* de campaña es reanudable: persiste cada resultado atómicamente al completar una instancia (*checkpoint* por instancia×semilla). Ante una interrupción, al rearrancar se detectan los resultados ya persistidos y se procesan únicamente las instancias pendientes, sin duplicar filas ni perder determinismo.

3.5.5. Entorno de experimentación

Como parte de las garantías de reproducibilidad, se detalla el entorno de ejecución sobre el que se obtuvieron los resultados. El grueso de las campañas se ejecutó en un

portátil convencional (Apple M3 Pro), coherente con el enfoque de bajo coste dirigido a las PYMEs y sin cómputo en la nube; la campaña de calidad guiada (EXP-007) y la reejecución en entorno único para la comparación de tiempo se realizaron en un servidor del laboratorio (Intel Core i7-13700KF, Ubuntu 22.04) facilitado por la tutela. La Tabla 3.1 recoge la configuración de *hardware* y *software* de ambas máquinas. El servidor dispone además de una GPU (NVIDIA GeForce RTX 4070 Ti) que el algoritmo no utiliza: todo el cómputo es en CPU mediante NumPy. Como las campañas se repartieron entre las dos máquinas, la comparación principal se hace por RPD, que es independiente del *hardware*; la comparación de operadores se reejecutó además por completo en un entorno único (el mismo servidor, con igual presupuesto), y esa reejecución reprodujo exactamente las cifras de calidad obtenidas en el portátil, lo que verifica empíricamente la reproducibilidad del trabajo. El coste temporal exige un cuidado adicional, porque no basta con emplear la misma máquina: las mediciones deben ser pareadas en el tiempo, por lo que se aborda con un experimento específico cuyos resultados se analizan en la Sección 4.6.3.

Componente	Portátil	Servidor
Procesador (CPU)	Apple M3 Pro (12 núcleos)	Intel Core i7-13700KF (16 núcleos, 24 hilos)
Memoria RAM	36 GB	32 GB
Almacenamiento	SSD 460 GB	SSD 2 TB
Sistema operativo	macOS 26.5.1 (arm64)	Ubuntu 22.04.5 LTS (x86-64, kernel 6.8)
Lenguaje	Python 3.12.10	Python 3.12.13
Cálculo numérico	NumPy 2.3.3	NumPy 2.3.3
Pruebas y calidad	pytest, ruff, black	pytest, ruff, black

TABLA 3.1. ENTORNO DE *HARDWARE* Y *SOFTWARE* DE LA EXPERIMENTACIÓN (PORTÁTIL Y SERVIDOR).

Los tiempos absolutos se registran únicamente como indicador de viabilidad: al haberse ejecutado las campañas en equipos de uso general, y en ventanas temporales distintas, no son comparables entre sí, y la comparación de calidad se apoya en la RPD (Sección 3.5.2). La comparación del coste de cómputo entre operadores, en cambio, sí se aborda con rigor mediante mediciones pareadas en un entorno único, y se analiza en la Sección 4.6.3.

4. RESULTADOS EXPERIMENTALES

Este capítulo presenta los resultados obtenidos en las campañas experimentales descritas en la Sección 3.5. La evaluación se organizó como una secuencia de experimentos con propósitos complementarios: primero se validó el *baseline* constructivo NEH sobre ambos *benchmarks*; después se calibró el presupuesto de parada del *Iterated Greedy*; a continuación se validó una aceleración de la evaluación de inserciones que hace viable la campaña a gran escala; y, finalmente, se ejecutó la campaña comparativa del IG frente al *baseline*. El capítulo se cierra con la aportación (destrucción inteligente) y el análisis comparativo.

4.1. Resumen de las campañas experimentales

Antes de presentar los resultados conviene explicitar qué persigue cada experimento y con qué medios, para que las cifras posteriores se interpreten en su contexto. La Tabla 4.1 resume las ocho campañas; cada una responde a una pregunta concreta del desarrollo. El identificador EXP-NNN enlaza cada resultado con su configuración y sus datos crudos (trazabilidad completa en el repositorio del proyecto). Conviene advertir que ese identificador es una etiqueta de trazabilidad y no un orden de lectura: la tabla y las secciones siguen el orden narrativo del capítulo, en el que EXP-005 precede a EXP-004 porque la aceleración que valida el primero es la que hace viable la campaña del segundo.

En síntesis, la lógica es incremental. Las cinco primeras campañas construyen y validan el marco de comparación: EXP-001 y EXP-002 fijan la referencia contra la que se mide toda mejora; EXP-003 elige el presupuesto de cómputo con el mejor compromiso calidad/coste; EXP-005 garantiza que la optimización de rendimiento no altera los resultados y cuantifica cuánto acelera; y EXP-004 responde a la pregunta central (¿mejora el IG al *baseline*?) con réplicas y análisis de robustez. Las tres últimas se dedican a la aportación: EXP-006 fija su único parámetro propio, EXP-007 la contrasta con la destrucción aleatoria y con una variante puramente voraz sobre el *benchmark* completo, y EXP-008 explica el ahorro de tiempo que esa comparación revela. Las secciones siguientes desarrollan cada uno.

4.2. Baseline NEH (EXP-001 y EXP-002)

La heurística NEH se ejecutó sobre la totalidad de ambos benchmarks (120 instancias Taillard y 480 instancias VRF). Al ser determinista, basta una única ejecución por instancia. Los resultados constituyen la línea base contra la que se evalúan las metaheurísticas.

ID	Objetivo	Algoritmo	Datos	Métrica
EXP-001	Validar el <i>baseline</i> constructivo	NEH	Taillard (120)	RPD vs. UB
EXP-002	Extender el <i>baseline</i> al <i>benchmark</i> de alta dureza	NEH	VRF (480)	RPD vs. mejor conocido
EXP-003	Calibrar el presupuesto de parada del IG	IG ($d=4$, $T_p=0,4$, con búsqueda local)	8 Taillard + 7 VRF <i>Small</i>	RPD vs. presupuesto
EXP-005	Validar la aceleración y medir el <i>speedup</i>	IG <i>naive</i> vs. acelerado	Subconjunto de EXP-003	<i>makespan</i> idéntico; <i>speedup</i>
EXP-004	Comparar el IG con el <i>baseline</i> NEH	IG (presup. 500, 5 semillas)	Taillard + VRF	RPD vs. NEH
EXP-006	Calibrar el tamaño de la RCL del operador guiado	IG con destrucción guiada (<i>idle-rcl</i>)	23 instancias (las de EXP-003 + 8 discriminantes)	RPD vs. α
EXP-007	Contrastar las tres vías de destrucción (ablaación)	IG: aleatoria, voraz y guiada	Taillard + VRF (600)	RPD; Friedman y Nemenyi
EXP-008	Explicar el ahorro de tiempo de la guía	IG: aleatoria frente a guiada, medición pareada	36 instancias estratificadas	Pasadas, evaluaciones y tiempo

TABLA 4.1. RESUMEN DE LOS EXPERIMENTOS: OBJETIVO, ALGORITMO, DATOS Y MÉTRICA. UB (*UPPER BOUND*): MEJOR *MAKESPAN* PUBLICADO PARA LA INSTANCIA.

4.2.1. Resultados sobre Taillard

La campaña EXP-001 se ejecutó sobre las 120 instancias Taillard. El RPD medio global (frente al *upper bound* de 1993) fue de 3,01 % ($\sigma = 1,91$ %; mín. 0,08 %; máx. 8,00 %). El invariante de sanidad (ningún *makespan* inferior al *lower bound*) se verificó en las 120 instancias (0 violaciones).

La Tabla 4.2 desglosa el RPD por grupo de tamaño $n \times m$.

$n \times m$	RPD medio (%)	RPD mín. (%)	RPD máx. (%)
20×5	3,25	0,44	7,22
20×10	4,59	3,39	6,19
20×20	3,73	1,75	5,53
50×5	0,73	0,17	1,42
50×10	4,57	2,76	5,83
50×20	6,06	4,97	8,00
100×5	0,48	0,08	1,52
100×10	2,17	1,25	3,86
100×20	4,28	3,28	5,02
200×10	1,23	0,68	2,12
200×20	3,33	2,23	4,36
500×20	1,73	1,22	2,26

TABLA 4.2. RPD DEL NEH SOBRE TAILLARD (EXP-001), POR GRUPO $N \times M$.

Se observa un patrón coherente con la literatura. Cada grupo $n \times m$ reúne las diez instancias con el mismo número de trabajos n y de máquinas m (Sección 2.1). El RPD medio empeora al aumentar las máquinas: 1,49 % con $m = 5$, 3,14 % con $m = 10$ y 3,82 % con $m = 20$. En cambio, mejora al aumentar los trabajos con m fijo: con $m = 10$ desciende del 4,59 % en las instancias de 20 trabajos al 1,23 % en las de 200. Esta segunda tendencia admite una excepción, el salto de 20×20 a 50×20 (del 3,73 % al 6,06 %, el peor grupo del *benchmark*), tras el cual el descenso se reanuda hasta el 1,73 % de 500×20 . El NEH degrada, por tanto, su calidad relativa a medida que se añaden máquinas y la recupera a medida que se añaden trabajos.

EXP-001 deja fijada, por tanto, la referencia sobre Taillard: un RPD medio del 3,01 % con el que contrastar toda mejora posterior, obtenido en una única ejecución por instancia y sin violar en ningún caso la cota inferior. Su interés no está en la calidad alcanzada, modesta como corresponde a una heurística constructiva, sino en que reproduce el comportamiento descrito en la literatura: es la comprobación de que la implementación es correcta antes de construir nada encima de ella.

4.2.2. Resultados sobre VRF

La campaña EXP-002 se completó sobre las 480 instancias VRF (240 *Small* + 240 *Large*). El RPD medio global (frente al mejor *makespan* conocido) fue de 3,59 % ($\sigma = 1,87$ %). Por subconjunto:

- *Small*: RPD medio 3,84 % ($\sigma = 2,32$ %).
- *Large*: RPD medio 3,33 % ($\sigma = 1,22$ %; mín. 0,77 %; máx. 6,42 %).

Se verificó cobertura completa (480 instancias resueltas, 0 RPD negativos) y 0 *makespan* ≤ 0 . El RPD medio sobre VRF es superior al obtenido sobre Taillard (3,59 % frente a 3,01 %), lo que respalda empíricamente la mayor dureza del *benchmark* de 2015 y su superior poder de discriminación estadística [6]. La Figura 4.1 desglosa esa comparación por número de máquinas.

La lectura de la figura exige dos precisiones. La primera es que los dos *benchmarks* no cubren los mismos tamaños: Taillard se limita a $m \in \{5, 10, 20\}$ máquinas, mientras VRF añade los valores 15, 40 y 60 (Sección 2.9). En esos tres valores no aparece barra de Taillard porque no existe ninguna instancia suya con ese número de máquinas, no porque su RPD sea nulo. La segunda es que esa diferencia de cobertura explica buena parte del margen global: al restringir la comparación a los tres valores de m comunes, la distancia entre ambos *benchmarks* baja de 0,58 a 0,23 pp, y en dos de ellos ($m = 5$ y $m = 20$) el RPD del NEH resulta incluso algo menor en VRF. Las 220 instancias de VRF con $m \in \{15, 40, 60\}$, en cambio, promedian un 4,00 %. La mayor dureza de VRF no se manifiesta como una penalización uniforme instancia a instancia, sino en que el conjunto incorpora configuraciones de mayor dificultad, con más máquinas y hasta 800 trabajos, que Taillard no alcanza.

EXP-002 completa así la línea base sobre el *benchmark* que decide la evaluación: un RPD medio del 3,59 % en 480 instancias, con cobertura total y sin anomalías. Confirmada la mayor dureza de VRF y localizadas las configuraciones en las que el NEH se degrada, queda establecida la referencia frente a la que se miden el IG y la aportación en el resto del capítulo.

4.3. Calibración del presupuesto del IG (EXP-003)

Antes de lanzar la campaña completa fue necesario fijar el presupuesto de parada del IG: cuántas iteraciones conceder a cada ejecución. Un presupuesto insuficiente deja calidad sobre la mesa; uno excesivo desperdicia cómputo sin mejora apreciable, en contra del enfoque de bajo coste del trabajo. El objetivo de EXP-003 es localizar el punto donde el RPD deja de mejorar de forma apreciable (la *rodilla* de la curva).

Se barrió el presupuesto en $\{50, 100, 200, 500, 1000, 2000, 5000\}$ iteraciones sobre un subconjunto representativo de 15 instancias (8 de Taillard, una por grupo $n \times m$ tratable,

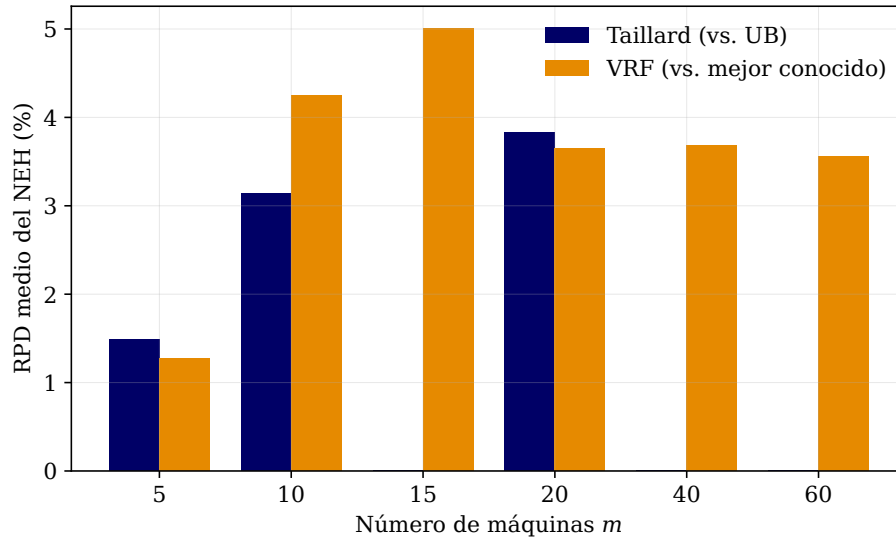


Fig. 4.1. RPD medio del NEH por número de máquinas (EXP-001/002): Taillard (frente al *upper bound*) y VRF (frente al mejor *makespan* conocido). El RPD sube con fuerza al pasar de $m = 5$ a $m = 10$ en ambos *benchmarks* y se mantiene alto después, con el máximo de VRF en $m = 15$. Taillard no incluye instancias con $m \in \{15, 40, 60\}$, de ahí la ausencia de barra en esos valores.

y 7 de VRF *Small*), con 3 réplicas por instancia (semillas 42, 123, 456). La Tabla 4.3 muestra el RPD medio y su mejora marginal.

Presup. (iter.)	RPD medio (%)	Δ marginal (pp)	Tiempo/ejec. (s)
50	0,488	—	7,3
100	0,388	0,100	14,5
200	0,281	0,107	30,4
500	0,185	0,096	76,5
1000	0,155	0,030	143,1
2000	0,130	0,025	278,4
5000	0,107	0,024	738,4

TABLA 4.3. CALIBRACIÓN DEL PRESUPUESTO DEL IG
(EXP-003): RPD MEDIO Y MEJORA MARGINAL.

El grueso de la mejora se concentra entre 50 y 500 iteraciones (RPD de 0,49 % a 0,18 %). A partir de 500, la mejora marginal cae por debajo de 0,1 pp mientras el coste sigue creciendo casi linealmente (de ~ 77 s a ~ 738 s por ejecución al pasar de 500 a 5000). Se elige por tanto un presupuesto de 500 iteraciones, que equilibra calidad y coste, coherente con el objetivo del trabajo. Sobre el mismo subconjunto, el IG a 500 iteraciones reduce el RPD medio del NEH de 3,19 % a 0,18 % (factor ~ 17), lo que anticipa la mejora que confirmará EXP-004. La Figura 4.2 representa la curva de RPD frente al presupuesto, donde se aprecia la rodilla en 500 iteraciones.

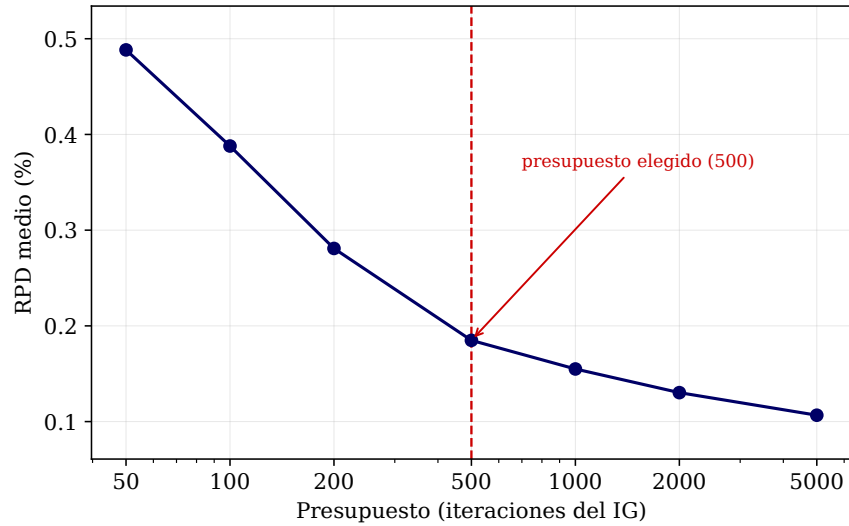


Fig. 4.2. Calibración del presupuesto del IG (EXP-003): RPD medio frente al presupuesto de iteraciones (escala logarítmica); la rodilla en 500 marca el compromiso calidad/coste adoptado.

EXP-003 se cierra, por tanto, con la decisión de diseño que gobierna todas las campañas posteriores: 500 iteraciones como presupuesto único, adoptado no por convención sino porque es el punto donde la mejora marginal cae por debajo de 0,1 pp mientras el coste por ejecución sigue creciendo de forma casi proporcional al presupuesto. Multiplicar el cómputo por diez a partir de ahí recorta apenas 0,08 pp de RPD, un intercambio incompatible con el enfoque de bajo coste que persigue el trabajo.

4.4. Aceleración de la evaluación de inserciones (EXP-005)

La evaluación repetida del *makespan* es la operación más costosa del IG. La implementación directa recalcula el *makespan* en cada posición de inserción, con un coste de $O(n^3m)$ por pasada de búsqueda local, lo que resulta inviable en las instancias grandes de VRF. Para hacer factible la campaña se incorporó la aceleración de Taillard [9], que precomputa matrices de tiempos de finalización “hacia adelante” y “hacia atrás” y evalúa todas las inserciones en $O(n^2m)$. EXP-005 verifica que esta optimización *no altera los resultados* y mide cuánto acelera.

Re-ejecutando el barrido de EXP-003 con la versión acelerada se obtuvieron 315 de 315 *makespan* idénticos a los de la versión directa: la aceleración es una mejora de rendimiento pura, por lo que las conclusiones de EXP-003 siguen siendo válidas. El *speedup* crece con el tamaño de la instancia, como predice el cambio de complejidad (Tabla 4.4).

Ambos barridos —el directo (EXP-003) y el acelerado (EXP-005)— se ejecutaron en el equipo portátil descrito en la Tabla 3.1, por lo que el *speedup* es un cociente entre medidas tomadas en la misma máquina y con las mismas instancias y semillas. Conviene precisar el alcance de estas cifras: los tiempos absolutos del portátil se emplean aquí úni-

Instancia	$n \times m$	Directo (s)	Acelerado (s)	<i>Speedup</i>
tai20_5_0	20 × 5	5,71	0,84	6,8×
tai20_20_0	20 × 20	18,53	2,92	6,3×
VFR40_5_1	40 × 5	17,68	0,98	18,0×
tai50_20_0	50 × 20	139,77	7,32	19,1×
VFR60_10_1	60 × 10	120,98	5,09	23,7×
tai100_10_0	100 × 10	340,53	7,72	44,1×
tai100_5_0	100 × 5	255,47	3,03	84,4×

TABLA 4.4. ACELERACIÓN (EXP-005): TIEMPO POR EJECUCIÓN A PRESUPUESTO 500 Y *SPEEDUP*. TIEMPOS MEDIDOS EN EL EQUIPO PORTÁTIL DESCRITO EN EL ENTORNO DE EXPERIMENTACIÓN; EXP-003 Y EXP-005 SE EJECUTARON EN LA MISMA MÁQUINA, DE MODO QUE EL COCIENTE ES HOMOGÉNEO.

camente como *orden de magnitud* (para justificar la viabilidad de la campaña), no como base de comparación entre operadores. La gestión dinámica de frecuencia de un equipo de uso general introduce una variabilidad que no permite sostener diferencias de pocos puntos porcentuales; por ello la comparación de tiempos entre la destrucción aleatoria y la guiada se rehízo por completo en el servidor del laboratorio, en un entorno único (Sección 4.6.3).

La aceleración pasa de $\sim 6\times$ en las instancias de 20 trabajos a $40\text{--}84\times$ en las de 100, lo que hace factible la campaña. Con la versión acelerada, una réplica de VRF *Small* tarda menos de 8 s; en VRF *Large* se midieron 47,9 s (200×20) y 138,3 s (400×20) a presupuesto 500, de modo que las instancias mayores (600×40 , 800×60) siguen costando del orden de decenas de minutos por réplica. Por ello la campaña sobre VRF *Large* se planifica por tramos (véase la Sección 4.5). La Figura 4.3 contrasta el coste por ejecución de ambas implementaciones frente al tamaño de la instancia.

EXP-005 cumple sus dos objetivos. Establece, en primer lugar, la equivalencia funcional: los 315 *makespan* de la versión acelerada son idénticos a los de la directa, de modo que la optimización no compra velocidad a cambio de calidad y las conclusiones de EXP-003 siguen siendo válidas. Cuantifica, en segundo lugar, una ganancia que crece con el tamaño de la instancia, de $6\text{--}9\times$ en las de 20 trabajos a $44\text{--}84\times$ en las de 100, tal como predice el paso de $O(n^3m)$ a $O(n^2m)$. Sin esta aceleración la campaña sobre VRF *Large* no habría sido viable en el tiempo disponible: EXP-005 es, en ese sentido, la condición que hace posible el resto del capítulo.

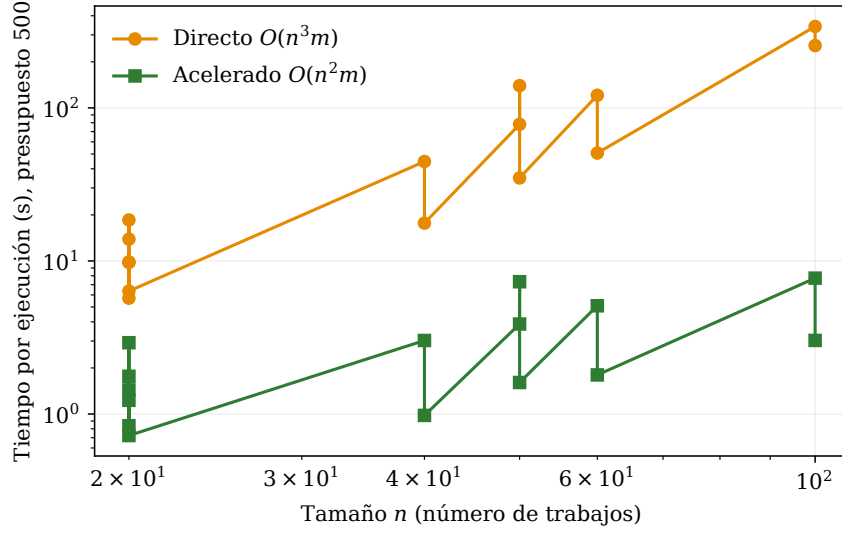


Fig. 4.3. Coste por ejecución frente al tamaño de la instancia a presupuesto 500 (EXP-005), implementación directa ($O(n^3m)$) frente a la acelerada ($O(n^2m)$); escala log-log. La divergencia de las curvas ilustra el cambio de complejidad.

4.5. Iterated Greedy (EXP-004)

El experimento central, EXP-004, responde a la pregunta que motiva la metaheurística: ¿mejora el *Iterated Greedy* de forma consistente al *baseline* constructivo? Para responderla con robustez se ejecuta el IG con réplicas multi-semilla y se compara su RPD medio con el del NEH grupo a grupo. La campaña EXP-004 se ejecutó con el IG clásico ($d = 4$, $T_p = 0,4$, búsqueda local activada) y un presupuesto de 500 iteraciones (calibrado en EXP-003), con 5 réplicas por instancia (semillas 42, 123, 456, 789, 2025). La campaña se completó sobre la totalidad de ambos *benchmarks*: Taillard (120 instancias) y VRF (480, *Small* + *Large*); esta última se ejecutó por tramos reanudados dada su carga (instancias de hasta 800×60), según la factibilidad medida en EXP-005.

4.5.1. Resultados globales

Sobre las 600 instancias comunes con el *baseline*, el *Iterated Greedy* reduce el RPD medio del 3,47 % que obtiene NEH al 0,60 %, una mejora de 2,87 puntos porcentuales. La ganancia se sostiene en ambos *benchmarks*: en Taillard el RPD baja del 3,01 % al 0,29 %, y en VRF —el conjunto de alta dureza, con sus 480 instancias *Small* y *Large*— del 3,59 % al 0,68 %. En términos relativos, el IG divide el error del *baseline* por un factor de 5,7 en el conjunto global, resultado de un factor de 10,4 sobre Taillard y de 5,3 sobre VRF: la reducción relativa es mayor sobre el *benchmark* más fácil, donde el IG llega a rozar las mejores soluciones publicadas. La mejora se produce además en todos los grupos de tamaño (Sección 4.5.2), lo que confirma lo que anticipaba la calibración (EXP-003).

4.5.2. Monotonía y sanidad

El IG mejora al NEH en los 51 grupos $n \times m$ comunes (0 violaciones de monotonía), desde 10×5 hasta 800×60 . El invariante $makespan \geq LB$ (*lower bound*, cota inferior) se verificó en vivo durante toda la campaña (0 anomalías). El IG batió el *upper bound* de Taillard (1993) en 5 instancias (*tai100_5_7*, *tai100_5_8*, *tai100_5_9*, *tai200_20_1* y *tai20_5_4*; RPD entre $-0,29\%$ y $-0,08\%$ frente al UB), lo cual es legítimo dado que los UB de 1993 son mejorables por metaheurísticas modernas y el invariante $makespan \geq LB$ se mantiene. La Figura 4.4 compara el RPD medio del NEH y del IG por número de máquinas en ambos *benchmarks*.

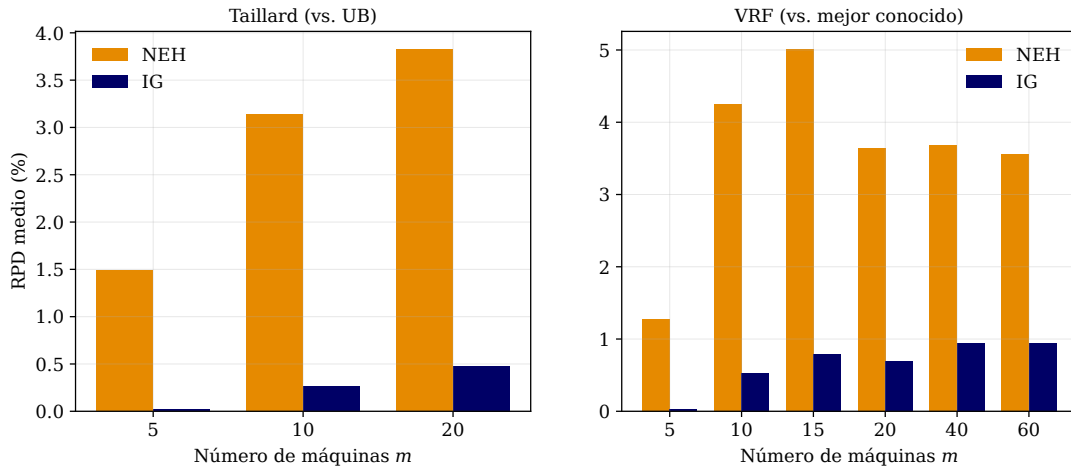


Fig. 4.4. RPD medio del NEH frente al IG por número de máquinas (EXP-004), separado por benchmark (Taillard, frente al UB; VRF, frente al mejor conocido). El IG reduce el RPD del *baseline* en todo el rango de m .

EXP-004 responde afirmativamente a la pregunta que lo motiva, y con margen amplio: el IG mejora al *baseline* en los 51 grupos de tamaño y divide su error por un factor de 5,7, sin una sola violación de los invariantes de sanidad en las 3.000 ejecuciones de la campaña. Queda con ello fijado el rival contra el que ha de medirse la aportación, y conviene reparar en qué clase de rival es: no una heurística débil, sino un IG clásico que ya opera muy cerca de las mejores soluciones publicadas, con un RPD del 0,29 % en Taillard. Esa cercanía acota de antemano el margen de mejora disponible y anticipa el efecto suelo que se documenta en la sección siguiente.

4.6. Destrucción inteligente

Esta sección evalúa la aportación del trabajo: el operador de destrucción guiada por tiempo muerto (*idle-RCL*). Antes de contrastarlo con la destrucción aleatoria del IG clásico es preciso fijar su único parámetro propio, el tamaño de la lista restringida de candidatos (RCL), tarea a la que se dedica la calibración que sigue; la comparación de calidad y eficiencia frente al *baseline* se presenta a continuación.

4.6.1. Calibración del parámetro α (EXP-006)

El operador de destrucción guiada construye en cada iteración una RCL con las posiciones de mayor tiempo muerto y extrae de ella los d trabajos que elimina. El tamaño de esa lista se fija como $\lceil \alpha n \rceil$ (acotado inferiormente por d), de modo que el parámetro $\alpha \in (0, 1]$ regula el compromiso entre explotación y diversificación: un α pequeño produce una RCL casi voraz (muy concentrada en los mayores tiempos muertos y propensa al estancamiento), mientras que un α grande la ensancha hasta aproximarse a la destrucción aleatoria, diluyendo la guía. Para justificar su valor en lugar de fijarlo a mano, se calibró empíricamente (EXP-006).

Se barrió $\alpha \in \{0,05, 0,10, 0,20, 0,30, 0,40, 0,50\}$ sobre 23 instancias de Taillard y VRF *Small* —las 15 del subconjunto de calibración de EXP-003 más 8 añadidas de mayor poder discriminante ($m \geq 10, n \leq 100$)— con 3 réplicas por instancia (semillas 42, 123, 456) y el presupuesto de 500 iteraciones fijado en EXP-003, para un total de 414 ejecuciones. La Tabla 4.5 recoge el RPD medio por valor de α .

α	RPD medio (%)	Δ marginal (pp)
0,05	0,987	—
0,10	0,818	0,169
0,20	0,557	0,261
0,30	0,390	0,167
0,40	0,382	0,008
0,50	0,369	0,013

TABLA 4.5. CALIBRACIÓN DEL TAMAÑO DE LA RCL (EXP-006):
RPD MEDIO POR α .

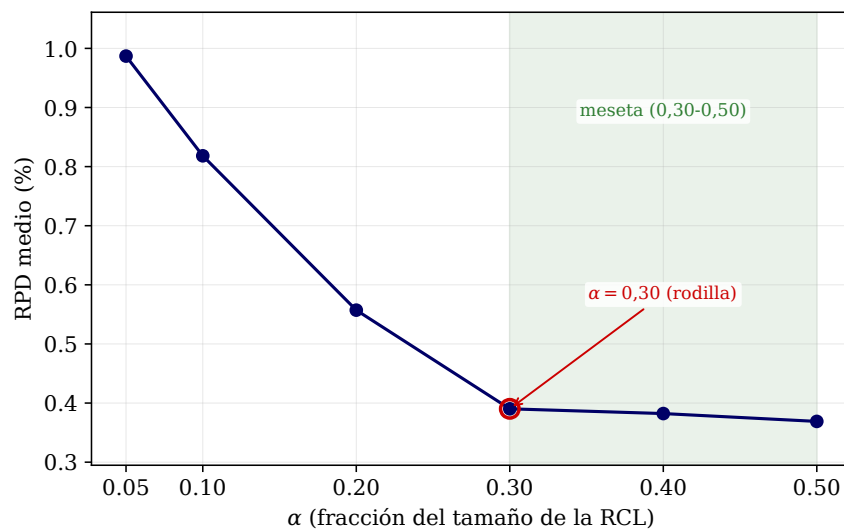


Fig. 4.5. Calibración de α (EXP-006): RPD medio frente al tamaño de la RCL. Descenso pronunciado hasta 0,30 y meseta posterior (0,30–0,50); se fija $\alpha = 0,30$ en la rodilla.

Como se aprecia en la Figura 4.5, el RPD medio decrece con fuerza hasta $\alpha = 0,30$ y, a partir de ahí, se estabiliza en una *meseta*: los valores 0,30, 0,40 y 0,50 difieren en menos de 0,02 pp, muy por debajo de la desviación típica entre semillas ($\sigma \approx 0,4$). Más allá de 0,30, por tanto, la elección de α es indistinguible dentro del ruido experimental. Esta apreciación visual se confirma con un contraste estadístico. Tomando como bloques las 23 instancias del subconjunto (cada una promediada sobre sus tres semillas) y comparando los tres niveles de la meseta mediante la prueba de Friedman, el mismo procedimiento no paramétrico empleado en la campaña comparativa (Sección 4.6.2), no se detectan diferencias significativas entre $\alpha \in \{0,30, 0,40, 0,50\}$ ($\chi^2 = 0,283$, $p = 0,868$). La meseta es, por tanto, estadísticamente plana: cualquiera de los tres valores es defendible y la decisión debe apoyarse en criterios distintos de la calidad media. Aunque el mínimo numérico se alcanza en 0,50, ese valor cae en el extremo de la rejilla y su posición dentro de la meseta resulta inestable frente a la muestra de instancias, por lo que no se adopta. Se fija $\alpha = 0,30$ —el punto donde termina el descenso pronunciado, la *rodilla* de la curva—: es un valor estable, corresponde a una RCL más selectiva (esto es, una destrucción más guiada y algo menos costosa, coherente con el enfoque de bajo coste del trabajo) y su diferencia de calidad frente a valores mayores es despreciable.

Conviene subrayar que α se calibra únicamente sobre este subconjunto reducido; el valor elegido se evalúa después sobre el *benchmark* completo en la campaña comparativa (Sección 4.6.2), de manera que la aportación se mide mayormente sobre instancias no empleadas para ajustar el parámetro. VRF *Large*, en particular, queda por completo fuera de la calibración, de modo que el resultado sobre el que descansa la aportación se obtiene en instancias enteramente nuevas para el operador.

EXP-006 fija, en suma, el único parámetro propio de la aportación con un criterio explícito y reproducible: $\alpha = 0,30$, en la rodilla de la curva y en el extremo selectivo de una meseta que el contraste de Friedman declara estadísticamente plana. La calibración deja además una lectura que conviene retener para interpretar la comparación que sigue: el operador es muy sensible a α por debajo de 0,30, pues el RPD se multiplica por dos y medio al reducirlo hasta 0,05, y prácticamente insensible por encima. Lo determinante no es, por tanto, el valor exacto del parámetro, sino que la RCL sea lo bastante ancha para preservar el muestreo aleatorio.

4.6.2. Comparación con la destrucción aleatoria

La aportación se evalúa en un doble eje: la eficiencia de búsqueda, esto es la convergencia del RPD frente al presupuesto, y la calidad sobre las instancias duras. El eje de eficiencia (EXP-007) reejecuta el barrido de presupuestos de EXP-003 con cada operador guiado y lo superpone a la curva del IG aleatorio (Figura 4.6). La destrucción voraz (*idle-greedy*) se estanca desde el presupuesto más bajo, con un RPD en torno al 1,26 % que no mejora al conceder más iteraciones. La guiada (*idle-rcl*) converge con la misma forma que la aleatoria, aunque a una meseta ligeramente peor sobre este subconjunto de

baja dificultad, donde el IG aleatorio ya opera cerca del suelo del RPD. El margen de la guía se espera, por tanto, en las instancias duras.

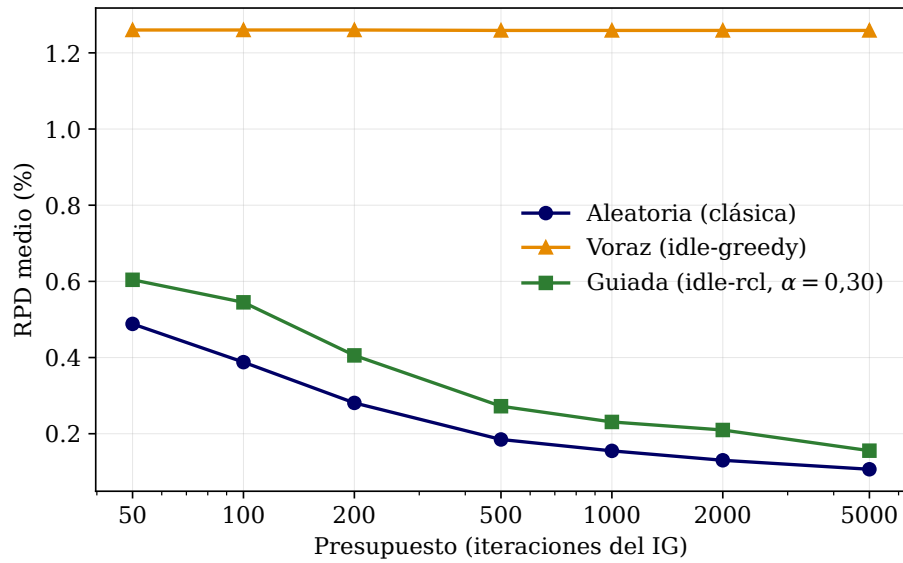


Fig. 4.6. Eje eficiencia (EXP-007): convergencia del RPD medio frente al presupuesto para la destrucción aleatoria, la voraz (*idle-greedy*) y la guiada (*idle-rcl*, $\alpha = 0,30$) sobre el subconjunto de calibración.

El eje de calidad contrasta las tres vías de destrucción —aleatoria (IG clásico), voraz (*idle-greedy*) y guiada (*idle-rcl*, la propuesta)— sobre las 600 instancias de ambos *benchmarks*, con cinco semillas por instancia y reutilizando como referencia el NEH y el IG clásico de EXP-004. La Tabla 4.6 recoge el resultado.

Operador	Taillard (%)	VRF <i>Small</i> (%)	VRF <i>Large</i> (%)	Global (%)	Rango medio
Aleatoria (IG clásico)	0,291	0,546	0,820	0,604	1,521
Voraz (<i>idle-greedy</i>)	1,615	2,032	2,261	2,040	2,918
Guiada (<i>idle-rcl</i> , $\alpha=0,30$)	0,314	0,641	0,808	0,643	1,562

TABLA 4.6. CALIDAD DE LAS TRES VÍAS DE DESTRUCCIÓN (EXP-007): RPD MEDIO POR *BENCHMARK* Y RANGO MEDIO DE FRIEDMAN, SOBRE 600 INSTANCIAS Y 5 SEMILLAS.

El test de Friedman detecta diferencias muy significativas ($\chi^2 = 758,1$, $p < 10^{-6}$) y el post-hoc de Nemenyi (diferencia crítica $CD = 0,14$) las ordena. La destrucción voraz es significativamente peor que las otras dos en los tres *benchmarks*: reincide en las mismas extracciones de alto tiempo muerto, no diversifica y converge a óptimos locales peores que los del propio azar, con un RPD del 2,04 % frente al 0,60 %. La guiada y la aleatoria quedan, en cambio, estadísticamente empatadas en el global (diferencia de rangos $0,04 < CD$). La ablación arroja así una lección de diseño central del trabajo: la lista restringida de candidatos y el muestreo uniforme no son un adorno, sino el ingrediente que hace aprovechable la guía por tiempo muerto.

Ese empate global esconde, no obstante, un comportamiento que depende de la dificultad de las instancias, como desglosa la Tabla 4.7.

Benchmark (n.º inst.)	ΔRPD (pp)	Gana / empata / pierde	Friedman (2 vías)
Taillard (120)	+0,023	48 / 22 / 50	$p = 0,86$ (n.s.)
VRF <i>Small</i> (240)	+0,095	43 / 80 / 117	$p < 10^{-5}$ (aleatoria mejor)
VRF <i>Large</i> (240)	-0,011	145 / 3 / 92	$p < 10^{-3}$ (guiada mejor)

TABLA 4.7. DESTRUCCIÓN GUIADA FRENTE A LA ALEATORIA, INSTANCIA A INSTANCIA (EXP-007). UN Δ RPD NEGATIVO INDICA QUE LA GUIADA MEJORA.

El signo de la diferencia se invierte con el tamaño del problema. En las instancias pequeñas y fáciles (Taillard y VRF *Small*), donde el IG aleatorio ya opera muy cerca del óptimo, concentrar la destrucción en las posiciones de mayor tiempo muerto resta diversificación sin apenas margen que ganar, y la guiada queda por detrás: en Taillard la diferencia es estadísticamente indistinguible de cero (Δ RPD de 0,023 pp, $p = 0,86$), mientras que en VRF *Small* la desventaja, pequeña en magnitud (0,095 pp), es lo bastante sistemática (pierde en 117 de las 240 instancias y gana solo en 43) para resultar significativa. La columna de empates mide ese *efecto suelo*: en 22 de las 120 instancias de Taillard y en 80 de las 240 de VRF *Small* ambos operadores alcanzan exactamente el mismo RPD medio, frente a solo 3 de 240 en VRF *Large*. En las instancias grandes y duras, en cambio, los huecos de inactividad son más numerosos y tienen más estructura que explotar, de modo que dirigir la eliminación hacia ellos focaliza la búsqueda donde sí hay mejora posible, y la guiada supera a la aleatoria de forma significativa. La destrucción inteligente rinde, por tanto, en el régimen para el que fue concebida: el de las instancias industriales de gran tamaño.

La magnitud de estas diferencias es, en todo caso, pequeña, del orden de centésimas de punto porcentual. Con 500 iteraciones y una búsqueda local que pule cada solución reconstruida, la destrucción fija el punto de partida de cada iteración, pero su efecto sobre el *makespan* final es limitado. La lectura práctica es que la guiada iguala a la aleatoria en el conjunto global y la mejora en el subconjunto duro (el de interés industrial) por un margen estrecho, pero estadísticamente significativo.

4.6.3. Coste de cómputo de los operadores

El segundo eje de la evaluación pregunta cuánto cuesta la guía. La cuestión no es ornamental: el operador propuesto sustituye un sorteo en $O(n)$ por un cálculo de puntuaciones de tiempo muerto en $O(nm)$ seguido de una ordenación, de modo que lo esperable a priori sería que resultase más lento. Responderla exigió una campaña propia, EXP-008, cuyo diseño conviene justificar porque de él depende la validez de la cifra que se reporta.

Las campañas de calidad de la destrucción aleatoria y de la guiada se midieron inicialmente en máquinas distintas, así que sus tiempos no eran comparables. La comparación

completa se reejecutó por ello en un entorno único, el mismo servidor y con idéntico presupuesto (Sección 3.5.5). Esa reejecución reprodujo exactamente las cifras de RPD obtenidas en el portátil, que coincidieron hasta la cuarta cifra decimal en las nueve combinaciones de operador y *benchmark*: la comparación de calidad es independiente del *hardware*, y la reejecución constituye en sí misma una comprobación de reproducibilidad del trabajo.

El entorno único no bastó, sin embargo, para el tiempo: aunque ambas campañas se ejecutaron en el mismo servidor, lo hicieron separadas por una semana, y ese desfase invalida la comparación. El diseño definitivo es, por tanto, una medición pareada: ambos operadores ejecutados consecutivamente sobre la misma instancia, en la misma máquina y en la misma ventana temporal, sobre una muestra estratificada de 36 instancias (una por grupo $n \times m$ de Taillard y de VRF *Large*) con tres semillas y el presupuesto de la campaña. Su validez se audita con un control interno que no exige conocer el estado de la máquina: el coste por evaluación de inserción ha de ser idéntico para los dos operadores, pues es la misma operación sobre los mismos datos. La medición pareada lo respeta —difiere en un 0,2 % en las instancias grandes—, mientras que la anterior lo violaba en un 25 %, lo que es materialmente imposible, de modo que sus cifras se descartan.

Sobre esa muestra, la destrucción guiada resulta un 4,5 % más rápida que la aleatoria de media, y lo es en 30 de las 36 instancias. La variante voraz es bastante más rápida que ambas, pero a costa de una calidad muy inferior.

Conviene precisar la incertidumbre de esa cifra, dado que el tiempo de ejecución (a diferencia del *makespan*) no es una magnitud determinista. La mediana del ahorro coincide con la media (4,5 %) y su intervalo de confianza al 95 % abarca del 2,7 % al 6,3 %, de modo que el ahorro es significativo (el intervalo excluye el cero) pero acotado a un solo dígito porcentual. La dispersión entre instancias es, sin embargo, notable: los casos individuales van desde un 18,6 % de ahorro hasta un 6,3 % de sobrecoste. El ahorro debe interpretarse, por tanto, como una tendencia agregada y no como una garantía instancia a instancia. Estrechar ese intervalo a la mitad exigiría cuadruplicar la muestra (unas 144 instancias), coste que no se consideró justificado para una métrica secundaria cuyo veredicto no está en discusión.

Queda por explicar a qué se debe ese ahorro, ya que ambos operadores ejecutan el mismo número de iteraciones. El coste de una iteración no es constante: lo domina la búsqueda local (el 97,8 % del tiempo de ejecución, según la instrumentación), que repite pasadas sobre la secuencia hasta que una pasada completa no encuentra ninguna mejora, de modo que su coste es proporcional al número de pasadas necesarias y este depende de la solución de partida. El resto de la iteración sí tiene coste fijo y, de hecho, la destrucción guiada resulta unas nueve veces *más* costosa que la aleatoria, pues debe calcular las puntuaciones de tiempo muerto, $O(nm)$, y ordenarlas, frente al sorteo en $O(n)$ de la aleatoria. El ahorro solo puede provenir, entonces, de que la guía reduzca el número de pasadas de búsqueda local: al extraer los trabajos que inducen más retardo frontal y recolocarlos

en la posición que minimiza el *makespan*, la solución reconstruida queda más cerca de un óptimo local del vecindario de inserción y deja menos movimientos de mejora por encontrar.

Los datos confirman ese mecanismo con precisión. La destrucción guiada necesita 2,05 pasadas por iteración frente a las 2,16 de la aleatoria (un 4,9 % menos, y menos en 31 de las 36 instancias), lo que supone un 4,8 % menos de evaluaciones de inserción y se traduce en el 4,5 % menos de tiempo ya citado. Las tres magnitudes coinciden, y lo hacen también instancia a instancia, como muestra la Figura 4.7: los puntos se alinean sobre la diagonal que representa la igualdad entre ahorro de trabajo y ahorro de tiempo. Queda así establecido no solo *que* la guía es más rápida, sino *por qué*, y en qué medida.

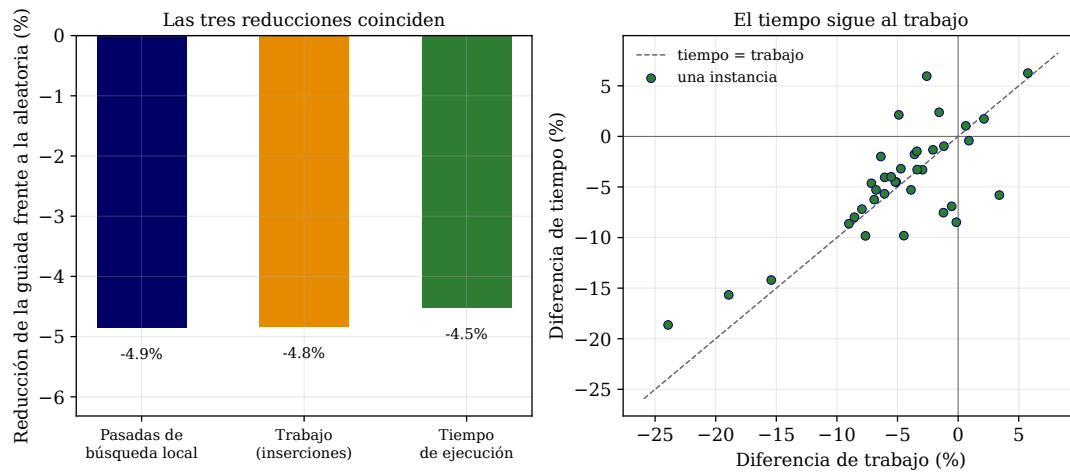


Fig. 4.7. Origen del ahorro de tiempo de la destrucción guiada (EXP-008, medición pareada sobre 36 instancias con tres semillas y presupuesto de 500 iteraciones). Izquierda: las tres reducciones frente a la destrucción aleatoria —pasadas de búsqueda local, evaluaciones de inserción y tiempo de ejecución— coinciden en torno al 5 %. Derecha: relación entre el ahorro de trabajo y el de tiempo en cada instancia; la diagonal discontinua representa la igualdad entre ambos.

El cruce de ambos ejes (calidad y coste) perfila el balance de la aportación. En VRF *Small* la guía es un compromiso desfavorable: cualquier ahorro de tiempo se paga con una pérdida de calidad significativa. En Taillard el balance es neutro en calidad, pues la diferencia no llega a ser distinguible de cero. En las instancias grandes de VRF *Large*, en cambio, resulta mejor en las dos dimensiones (RPD del 0,808 % frente al 0,820 %, y menos tiempo), si bien por márgenes estrechos en ambas. En el régimen de gran escala, por tanto, la aportación no se limita a igualar al IG clásico: obtiene soluciones algo mejores consumiendo algo menos de cómputo, con un mecanismo identificado y cuantificado.

4.7. Análisis comparativo

La Figura 4.8 sintetiza la comparación de calidad sobre las 600 instancias. El panel izquierdo muestra el RPD medio por *benchmark*: las tres metaheurísticas reducen drásti-

camente el RPD del *baseline* NEH, y las barras de la destrucción aleatoria y de la guiada son prácticamente indistinguibles (el empate global). El panel derecho, con la diferencia de RPD entre la guiada y la aleatoria por *benchmark*, revela el matiz que esas barras casi idénticas ocultan: el signo de la diferencia se invierte con la dificultad. En Taillard no es distinguible de cero; en VRF *Small* la guía queda por detrás (0,095 pp) y en VRF *Large* por delante (0,011 pp), en ambos casos de forma significativa. Conviene no leer esa significación como magnitud: la ventaja de la guía en VRF *Large* es la menor de las tres diferencias, casi nueve veces inferior a su desventaja en VRF *Small*, y resulta significativa por lo sistemática que es (gana en 145 de las 240 instancias y solo empata en 3), no por su tamaño. Es la traducción visual del efecto por dificultad descrito en la Sección 4.6.2.

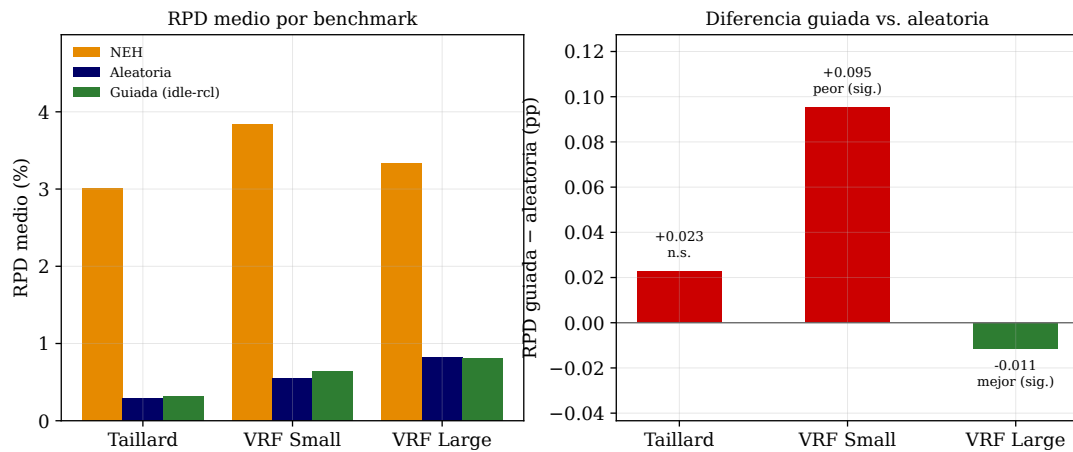


Fig. 4.8. Comparación de calidad (EXP-007, 600 instancias). Izquierda: RPD medio de NEH, la destrucción aleatoria (*random*) y la guiada (*idle-rcl*, $\alpha = 0,30$) por *benchmark*. Derecha: diferencia de RPD entre la guiada y la aleatoria por *benchmark* (los valores negativos indican que la guiada mejora); el signo se invierte con la dificultad de las instancias.

5. CONCLUSIONES Y TRABAJO FUTURO

Este capítulo recapitula el trabajo realizado y valora en qué medida se han alcanzado los objetivos planteados. En primer lugar se presentan las conclusiones, organizadas según los objetivos específicos enunciados en la introducción; a continuación se exponen las limitaciones que conviene tener presentes al interpretar los resultados; y, por último, se apuntan las líneas de trabajo futuro que abren la continuación del proyecto.

5.1. Conclusiones

Este trabajo se propuso diseñar, implementar y validar un algoritmo basado en *Iterated Greedy* (IG), de bajo coste computacional, para el PFSP con minimización del *makespan* y orientado a las PYMEs. A la luz de los resultados, se extraen las conclusiones siguientes, ordenadas según los objetivos específicos enunciados en la Sección 1.3.

Modelado del problema (objetivo 1). Se formalizó el PFSP como problema de optimización combinatoria NP-duro, con la recurrencia de cálculo del *makespan* y la descomposición del tiempo muerto (Capítulo 2), y se contextualizó su impacto operativo y económico en las PYMEs (Capítulo 1). El modelado sienta las bases metodológicas del resto del trabajo.

Línea base constructiva (objetivo 2). Se implementó la heurística NEH y se evaluó sobre la totalidad de ambos *benchmarks*: 120 instancias de Taillard (EXP-001) y 480 de VRF (EXP-002), con un RPD medio del 3,01 % y del 3,59 % respectivamente. El patrón observado en Taillard (el RPD empeora al aumentar las máquinas y mejora al aumentar los trabajos, salvo en el salto de 20×20 a 50×20) concuerda con la literatura, lo que valida la implementación como referencia fiable. En VRF el RPD medio es superior, si bien esa mayor dureza no se manifiesta como una penalización uniforme instancia a instancia, sino en que el conjunto incorpora configuraciones —más máquinas y hasta 800 trabajos— que Taillard no alcanza.

Metaheurística y calibración (objetivo 3). Se implementó el IG clásico y se calibraron sus parámetros de forma empírica y reproducible: el presupuesto de parada en 500 iteraciones, situado en la rodilla de la curva RPD–presupuesto (EXP-003), y el parámetro del operador guiado en $\alpha = 0,30$ (EXP-006). Además, se incorporó una aceleración de la evaluación de inserciones (EXP-005) que reduce el coste de $O(n^3m)$ a $O(n^2m)$ por pasada, con *speedups* de entre 6 y $84 \times$ y *makespan* idéntico al de la versión directa; esta optimización es la que hace viable la experimentación a gran escala en un equipo convencional.

El IG mejora sustancialmente a la línea base (objetivo 4). Sobre las 600 instancias comunes, el IG clásico reduce el RPD medio del NEH del 3,47 % al 0,60 % —una mejora de 2,87 puntos porcentuales, que divide el error del *baseline* por un factor de 5,7 (10,4 en

Taillard y 5,3 en VRF)— y supera al NEH en los 51 grupos $n \times m$ sin excepción (EXP-004). Queda confirmado el valor de la metaheurística frente a la heurística constructiva.

Aportación: destrucción guiada por tiempo muerto (objetivos 3 y 4). El análisis de ablación arroja una conclusión de diseño nítida: la eliminación puramente voraz de los trabajos de mayor tiempo muerto (la destrucción voraz, *idle-greedy*) se estanca (reincide en las mismas extracciones) y resulta netamente peor que la destrucción aleatoria, mientras que la variante con lista restringida de candidatos y muestreo uniforme (la destrucción guiada, *idle-RCL*, la propuesta) evita ese estancamiento. Esto es, la diversificación que aportan la RCL y el muestreo es imprescindible para que la guía por tiempo muerto sea aprovechable. En cuanto al rendimiento en calidad, sobre el *benchmark* completo (600 instancias) la destrucción guiada iguala a la aleatoria (empate estadístico según el test de Friedman con post-hoc de Nemenyi), pero con un comportamiento que depende de la dificultad: queda por detrás en VRF *Small*, de forma significativa aunque por un margen pequeño (donde el azar ya opera cerca del óptimo y la guía resta diversificación); no llega a distinguirse de la aleatoria en Taillard; y resulta significativamente superior en las instancias grandes y duras (VRF *Large*), en las que gana en 145 de 240 y donde el tiempo muerto ofrece más estructura que explotar. Ese resultado, que es el que sostiene la aportación, no está contaminado por la calibración: α se ajustó sobre un subconjunto reducido disjunto de VRF *Large*, de modo que la ventaja se mide sobre instancias que no se emplearon para fijar el parámetro. La mejora es modesta en magnitud (centésimas de punto porcentual, acotada por la búsqueda local y el presupuesto de cómputo), pero su dirección confirma la hipótesis del trabajo: la destrucción inteligente aporta precisamente en el régimen de gran escala, el de mayor interés industrial. No se alcanza, en cambio, una mejora de calidad decisiva y generalizada sobre la destrucción aleatoria, extremo que se retoma entre las limitaciones.

Coste de cómputo y balance global de la aportación (objetivo 4). Una medición pareada de ambos operadores —ejecutados consecutivamente sobre cada instancia, en la misma máquina y ventana temporal— muestra que la destrucción guiada es más rápida que la aleatoria en un 4,5 % a igualdad de presupuesto, siendo más rápida en 30 de las 36 instancias de la muestra. Es una ventaja modesta pero consistente, y también contraintuitiva, pues la fase de destrucción guiada es en sí misma unas nueve veces *más* costosa que el sorteo aleatorio, al tener que puntuar y ordenar las posiciones. Está, sobre todo, explicada: al dejar la solución reconstruida más cerca de un óptimo local del vecindario de inserción, la guía reduce el número de pasadas de búsqueda local necesarias (un 4,9 % menos), y es la búsqueda local la que domina el coste de cada iteración (el 97,8 % del tiempo), de modo que ese ahorro compensa con holgura la sobrecarga del operador. Las tres magnitudes —pasadas, evaluaciones de inserción y tiempo— coinciden en torno al 5 % y lo hacen instancia a instancia, de modo que queda establecido no solo que la guía es más rápida, sino por qué y en qué medida. Al cruzar ambos ejes, el balance resulta desfavorable en VRF *Small* (cualquier ahorro de tiempo se paga con la pérdida de calidad), neutro en calidad en Taillard y favorable en las dos dimensiones en las grandes de VRF

Large (mejor RPD y menos tiempo), aunque por márgenes estrechos. En el régimen de gran escala, por tanto, la aportación no se limita a igualar al IG clásico, si bien la mejora es de magnitud contenida.

Merece constancia, por su valor metodológico, que este resultado obligó a descartar una medición previa del tiempo. Aquella comparación se realizó en la misma máquina, pero con las dos campañas separadas por una semana, y no resiste una comprobación de consistencia: implicaba que una misma operación costaba un 25 % menos según qué operador la invocara, lo que es materialmente imposible (Sección 4.6.3). La lección es transferible: para comparar tiempos no basta con emplear la misma máquina, hace falta medición pareada; y la validez puede auditarse a posteriori comprobando que el coste por unidad de trabajo sea el mismo para todas las variantes comparadas.

Reproducibilidad y despliegue (objetivo 5). La solución se materializa en un paquete de código modular, *plug-and-play*, reproducible (semillas fijas, entorno y datos versionados) y sin dependencias comerciales (no requiere Gurobi ni CPLEX), ejecutable en equipos convencionales. La reproducibilidad no queda solo declarada, sino comprobada: al reejecutar la comparación completa en una máquina distinta, con otro sistema operativo y otra arquitectura (Tabla 3.1), el RPD coincidió hasta la cuarta cifra decimal en las nueve combinaciones de operador y *benchmark* (Sección 4.6.3). El repositorio incluye las instrucciones de instalación, compilación y ejecución, de modo que la solución puede clonarse y ponerse en marcha sin una infraestructura de despliegue industrial, en línea con el objetivo de producto mínimo viable orientado a las PYMEs. Los pasos concretos de reproducción se recogen en el Anexo A, pensado también para quien desee continuar o mejorar el trabajo.

5.2. Limitaciones

El trabajo presenta las siguientes limitaciones, que conviene tener presentes al interpretar sus resultados.

- **Alcance y magnitud de la mejora de la aportación.** La ventaja de la destrucción guiada en calidad es estadísticamente significativa pero modesta (centésimas de punto porcentual de RPD) y se limita al subconjunto de instancias grandes (VRF *Large*); en VRF *Small* es significativamente inferior, y por un margen mayor que su propia ventaja (0,095 pp frente a 0,011 pp); en Taillard la diferencia no llega a ser distinguible; y, en el conjunto completo, el resultado es de empate. La ventaja en coste de cómputo es igualmente modesta: un 4,5 % de ahorro de media, con un intervalo de confianza al 95 % que va del 2,7 % al 6,3 % de ahorro. Debe leerse, además, como tendencia agregada y no como garantía instancia a instancia, pues los casos individuales van desde un 18,6 % de ahorro hasta un 6,3 % de sobrecoste en el peor de ellos; y depende de la implementación concreta de la búsqueda local y

del lenguaje empleado, por lo que no debe extrapolarse sin más a otras implementaciones del IG.

- **Alcance de la medición del coste temporal.** La correspondencia entre el trabajo ahorrado y el tiempo ahorrado se ha establecido sobre una muestra estratificada de 36 instancias, no sobre el *benchmark* completo: medir de forma pareada las 600 instancias multiplicaría el coste de cómputo sin cambiar, previsiblemente, una conclusión que ya se sostiene instancia a instancia dentro de la muestra medida. Tampoco se ha medido de forma directa la comparación a igual tiempo en lugar de igual presupuesto de iteraciones. El efecto esperable puede, no obstante, acotarse con los datos disponibles: si la guía reinvirtiese íntegramente su ahorro de trabajo (4,8 %) en ejecutar más iteraciones, la mejora de calidad resultante sería del orden de 0,002 pp de RPD, según la pendiente de la curva de calibración en el entorno de las 500 iteraciones, donde incluso doblar el presupuesto mejora el RPD en solo 0,030 pp (Tabla 4.3). Esa magnitud queda muy por debajo de las diferencias de calidad en discusión: es unas cinco veces menor que la propia ventaja de la guía en VRF *Large* (0,011 pp) y casi cincuenta veces menor que su desventaja en VRF *Small* (0,095 pp), de modo que una comparación a igual tiempo no alteraría los veredictos alcanzados. La acotación es indicativa y no sustituye a la medición: la pendiente se estimó sobre el subconjunto de calibración, no sobre las instancias grandes.
- **Criterio de parada.** El presupuesto se expresa en iteraciones del IG y no mediante la fórmula temporal $n \cdot m \cdot t$ ms habitual en la literatura. La elección es deliberada y responde a la prioridad de reproducibilidad del trabajo (Sección 3.3.5): un presupuesto temporal hace depender el resultado de la velocidad de la máquina y de la eficiencia de la implementación, de modo que una misma corrida (misma instancia, misma semilla) deja de ser repetible entre equipos y un mismo valor de t concede a cada implementación un número distinto de iteraciones. El presupuesto en iteraciones, en cambio, asigna el mismo esfuerzo algorítmico a todas las variantes comparadas y hace la corrida regenerable, que es lo que exige la comparación de operadores que sostiene la aportación. La contrapartida se asume: la equivalencia exacta entre iteraciones, evaluaciones y milisegundos no se ha verificado contra el artículo original, de modo que los tiempos medidos sirven para comparar los operadores entre sí, pero no para situar el algoritmo frente a los tiempos publicados en la literatura.
- **Simplicidad deliberada del operador.** La guía se apoya en un único criterio (el tiempo muerto) y en un parámetro α estático; no se emplea selección adaptativa de operadores ni señales alternativas (por ejemplo, la ruta crítica). Es una decisión coherente con el enfoque de bajo coste y *plug-and-play*, pero acota el alcance frente a los métodos adaptativos del estado del arte.
- **Naturaleza de las instancias.** La evaluación se realiza sobre *benchmarks* académicas

cos (Taillard y VRF); no se dispuso de instancias industriales reales, de modo que la validez externa a un taller concreto se asume pero no se contrasta.

- **Profundidad del análisis estadístico.** La comparación se apoya en el RPD, en la monotonía frente al NEH y en el test de Friedman (global sobre las 600 instancias, con post-hoc de Nemenyi, y pareado por *benchmark*), pero no desciende al detalle: no se acompañan intervalos de confianza de la diferencia de RPD ni un desglose por grupo $n \times m$ de las tres variantes. Los datos crudos necesarios están disponibles, de modo que la limitación afecta al alcance del análisis y no a la cobertura experimental.

5.3. Líneas de trabajo futuro

A partir de lo desarrollado se abren varias líneas de continuación.

- **Selección adaptativa de operadores.** Sustituir la elección estática por un mecanismo que aprenda qué operador de destrucción aplicar en cada momento (por ejemplo, *Q-learning* tabular o un *multi-armed bandit* sobre un portafolio de operadores), extensión natural que conecta con las tendencias recientes del área.
- **Nuevos operadores de guía.** Explorar señales adicionales para dirigir la destrucción (como la ruta crítica o los bloques críticos) y variantes de muestreo, por ejemplo proporcional al tiempo muerto frente al muestreo uniforme.
- **Validez externa.** Incorporar instancias industriales reales, más allá de los *benchmarks* académicos (Taillard y VRF), para reforzar la validez externa de las conclusiones, en especial la ventaja observada en el régimen de gran escala.
- **Registrar los contadores de trabajo en toda la campaña.** Incorporar al algoritmo el recuento de pasadas de búsqueda local y de evaluaciones de inserción permitiría extender la correspondencia trabajo–tiempo al *benchmark* completo y, además, auditar la validez de cualquier medición de tiempo mediante el invariante del coste por unidad de trabajo, que en este trabajo permitió detectar una medición corrupta.
- **Comparación a igual tiempo y homologación del presupuesto.** Repetir el barrido de presupuestos sobre las instancias grandes tomando el tiempo como criterio de parada en lugar de las iteraciones. La acotación recogida entre las limitaciones sugiere que reinvertir el ahorro del 4,8 % apenas alteraría el RPD, por lo que el interés de esta línea no reside en confirmar ese margen, sino en caracterizar a ambos operadores bajo presupuestos temporales amplios, donde las curvas de convergencia podrían separarse más de lo que lo hacen en la rodilla adoptada en este trabajo. El mismo desarrollo permitiría establecer la equivalencia entre el presupuesto en iteraciones y la fórmula $n \cdot m \cdot t$ ms de la literatura, requisito para situar los tiempos obtenidos frente a los publicados.

- **Análisis estadístico más fino.** Explotar los datos crudos ya disponibles para ir más allá del contraste global: intervalos de confianza de la diferencia de RPD, desglose por grupo $n \times m$ de las tres variantes y contrastes restringidos a los regímenes en los que la guía muestra ventaja. Ampliar el número de semillas por instancia añadiría potencia al contraste. El objetivo es acotar con precisión la magnitud del efecto, y no solo su dirección, que es lo que este trabajo establece.
- **Aceleración adicional.** Compilación JIT con *Numba* u optimizaciones equivalentes para reducir el tiempo de cómputo en las instancias de mayor tamaño, sin alterar la lógica del algoritmo.
- **Métodos híbridos.** Combinar el IG con *path relinking* o con una búsqueda local más agresiva (Or-opt, 2-opt adaptado al *flow-shop*).
- **Despliegue en producción.** Envolver la solución en una interfaz ligera e integrarla con sistemas MES (*Manufacturing Execution System*) o ERP (*Enterprise Resource Planning*), y validarla con datos de producción reales en un entorno PYME.

6. MARCO REGULADOR

Este capítulo analiza el marco regulador aplicable al presente Trabajo de Fin de Grado. Al tratarse de un proyecto de carácter algorítmico y experimental (que desarrolla y evalúa software de optimización sobre instancias sintéticas de *benchmarks* públicos), el análisis se estructura en torno a tres ejes: la propiedad intelectual e industrial del software y de la memoria, los estándares técnicos adoptados y la protección de datos personales, cuya no aplicabilidad se justifica razonadamente.

6.1. Naturaleza del trabajo y alcance

El trabajo implementa y evalúa una heurística constructiva (NEH) y una metaheurística (*Iterated Greedy*) para el *Permutation Flow-Shop Scheduling Problem* (PFSP). Las instancias empleadas son matrices de tiempos de procesamiento sintéticas, tomadas de los conjuntos de referencia de Taillard y VRF. No se desarrolla un producto comercial, no se despliega en un entorno productivo real y no se tratan datos personales. En consecuencia, el marco regulador relevante se concentra en: (1) la propiedad intelectual e industrial del software y de la memoria, (2) los estándares técnicos aplicables al desarrollo y a la reproducibilidad, y (3) la protección de datos, que se analiza para justificar por qué no resulta aplicable.

6.2. Propiedad intelectual e industrial

El régimen de propiedad intelectual e industrial del trabajo afecta a cuatro elementos: el *software* de terceros en el que se apoya la implementación, el código propio desarrollado, los conjuntos de instancias empleados y la propia memoria. Esta sección revisa las licencias de cada uno y analiza la patentabilidad de la aportación algorítmica, con el objetivo de dejar claras las condiciones de uso, modificación y distribución de los resultados del proyecto.

6.2.1. Licencias del software de terceros

La implementación se apoya exclusivamente en software libre con licencias permisivas, lo que habilita su uso, modificación y distribución sin coste de licencia. La Tabla 6.1 recoge los componentes principales y sus licencias. El uso de un *stack* permisivo es coherente con el objetivo de bajo coste y reproducibilidad del proyecto: evita licencias comerciales (como las de Gurobi o CPLEX) y no introduce obligaciones de *copyleft* que condicionen la distribución del código propio.

Componente	Uso en el proyecto	Licencia
Python 3.12	Lenguaje base	PSF License Agreement
NumPy 2.3	Cálculo matricial del <i>makespan</i>	BSD-3-Clause
matplotlib	Figuras (capa de análisis, desarrollo)	Licencia matplotlib (PSF)
pytest	Marco de pruebas	MIT
ruff	<i>Linting</i> estático	MIT
black	Formateo automático	MIT

TABLA 6.1. COMPONENTES DE SOFTWARE DE TERCEROS Y SUS LICENCIAS.

6.2.2. Licencia del código propio

El código fuente desarrollado en este trabajo se publica bajo licencia BSD de tres cláusulas (BSD-3-Clause), elegida por su permisividad: permite a cualquier PYME usar, modificar y redistribuir el software sin obligaciones de *copyleft*, con la única condición de conservar el aviso de copyright y de no emplear el nombre del titular para promocionar productos derivados. Esta elección es coherente con el objetivo del proyecto (bajo coste, sin barreras), con las licencias de todas las dependencias empleadas (PSF, BSD-3, MIT), evitando incompatibilidades, y con la licencia que el grupo de investigación aplica al repositorio que aloja el trabajo. El repositorio se aloja en GitHub (grupo CAOS, UC3M) y permanece privado hasta la fecha de defensa. Los resultados experimentales (datos crudos de RPD) son generados por el propio software y no están sujetos a derechos de terceros.

6.2.3. Licencia de los conjuntos de instancias (*benchmarks*)

El trabajo emplea dos conjuntos de instancias ampliamente utilizados en la literatura:

- **Taillard (1993)** [27]: instancias de dominio público, redistribuidas en múltiples espejos académicos.
- **VRF (Vallada, Ruiz y Framinan, 2015)** [6]: distribuidas junto con la publicación para uso investigador.

El uso de ambos conjuntos es estrictamente académico y se realiza con atribución completa a los autores originales.

6.2.4. Licencia de la memoria

La memoria se publica bajo licencia Creative Commons Reconocimiento – No Comercial – Sin Obra Derivada (CC BY-NC-ND), que es la opción recomendada por la Universidad Carlos III de Madrid para los trabajos fin de estudios. Esta licencia permite

la difusión y la cita del trabajo con reconocimiento de su autoría, pero reserva los usos comerciales y la creación de obras derivadas.

6.2.5. Patentabilidad

Los métodos matemáticos y los programas de ordenador «como tales» están excluidos de patentabilidad en el marco europeo y español. La aportación de este trabajo (un operador de destrucción guiado por el tiempo muerto) constituye un método algorítmico y, por tanto, no es patentable como tal; su protección se articula mediante los derechos de autor sobre el código fuente y la memoria.

- **Convenio sobre la Patente Europea (CPE/EPC, Múnich 1973, rev. 2000), art. 52, apartados 2 y 3:** el apartado 2(a) excluye los «métodos matemáticos» y el apartado 2(c) los «programas de ordenador» de la noción de invención; el apartado 3 precisa que la exclusión opera «solamente en la medida en que la solicitud de patente europea o la patente europea se refiera a uno de esos elementos *considerado como tal*».
- **Ley 24/2015, de 24 de julio, de Patentes (España), art. 4, apartados 3 y 4:** el apartado 3 reproduce las exclusiones del CPE (métodos matemáticos, programas de ordenador, entre otros); el apartado 4 incorpora la misma limitación: la exclusión opera «solamente en la medida en que la solicitud de patente se refiera exclusivamente» al elemento «considerado como tal».

6.3. Protección de datos personales

El proyecto no trata datos de carácter personal: las instancias del PFSP son matrices de tiempos de procesamiento sintéticas (Taillard) o generadas de forma controlada (VRF), sin relación con personas físicas identificadas o identificables. En consecuencia, no resultan aplicables:

- **El Reglamento (UE) 2016/679** (Reglamento General de Protección de Datos, RGPD).
- **La Ley Orgánica 3/2018, de 5 de diciembre** (LOPDGDD).

La no aplicabilidad es una conclusión razonada (ausencia de datos personales en todo el flujo de datos del proyecto), no una omisión. Cabe señalar que, en un hipotético despliegue real en una PYME donde el sistema se alimentara de datos de producción que incluyeran información personal (por ejemplo, identificadores de operarios), sí entraría en el ámbito del RGPD; ese supuesto queda fuera del alcance del presente trabajo y se menciona como consideración para futuras líneas de despliegue.

6.4. Estándares técnicos

No existe una norma técnica de obligado cumplimiento para un algoritmo de *scheduling*, pero el desarrollo se acoge a estándares y buenas prácticas del sector que sostienen la calidad y la reproducibilidad:

- **Estilo y calidad de código:** PEP 8 (guía de estilo de Python), verificado con las herramientas *ruff* y *black*.
- **Reproducibilidad científica:** semilla aleatoria fija, entorno de ejecución capturado y datos crudos conservados íntegramente, conforme al protocolo de trazabilidad definido en el diseño experimental del trabajo.
- **Estilo bibliográfico:** IEEE, recomendado por la Biblioteca UC3M para el ámbito de ingeniería.

La adopción de estos estándares se relaciona directamente con los criterios de calidad técnica y justificación de la solución del trabajo.

6.5. Responsabilidad profesional y ética

El trabajo no plantea riesgos laborales ni de seguridad física: la salida del sistema es una recomendación de secuencia de producción (apoyo a la decisión), sin control directo de maquinaria. La responsabilidad operativa permanece en la organización que emplee la herramienta.

Respecto al uso de inteligencia artificial (IA) generativa, este se declara en el Anexo B, conforme a la normativa UC3M.

En materia de integridad académica, el trabajo no presenta fabricación de datos ni de citas; el control de plagio se realiza mediante Turnitin conforme a los procedimientos de la universidad.

6.6. Síntesis del marco regulador aplicable

El marco regulador de este TFG se resume en: (1) un *stack* de software libre con licencias permisivas, coherente con el objetivo de bajo coste; (2) una aportación algorítmica no patentable, protegida por derechos de autor; (3) la no aplicabilidad razonada del RGPD al no tratarse datos personales; y (4) la adopción de estándares técnicos de calidad y reproducibilidad. La memoria se publica bajo licencia CC BY-NC-ND y el uso de IA generativa se declara en el Anexo B.

7. ENTORNO SOCIO-ECONÓMICO Y PRESUPUESTO

Este capítulo presenta la planificación temporal del Trabajo de Fin de Grado y su presupuesto, analiza su impacto socio-económico esperado y establece su relación con los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030.

7.1. Planificación temporal del proyecto

El desarrollo del trabajo se organizó en fases sucesivas, algunas solapadas, que abarcan desde el encuadre inicial y la revisión del estado del arte hasta la implementación de los algoritmos, la campaña experimental y la redacción de la memoria. La Figura 7.1 presenta la planificación en un diagrama de Gantt: las fases hasta julio corresponden al trabajo ya realizado, y las posteriores, a la planificación prevista hasta la entrega y la preparación de la defensa.

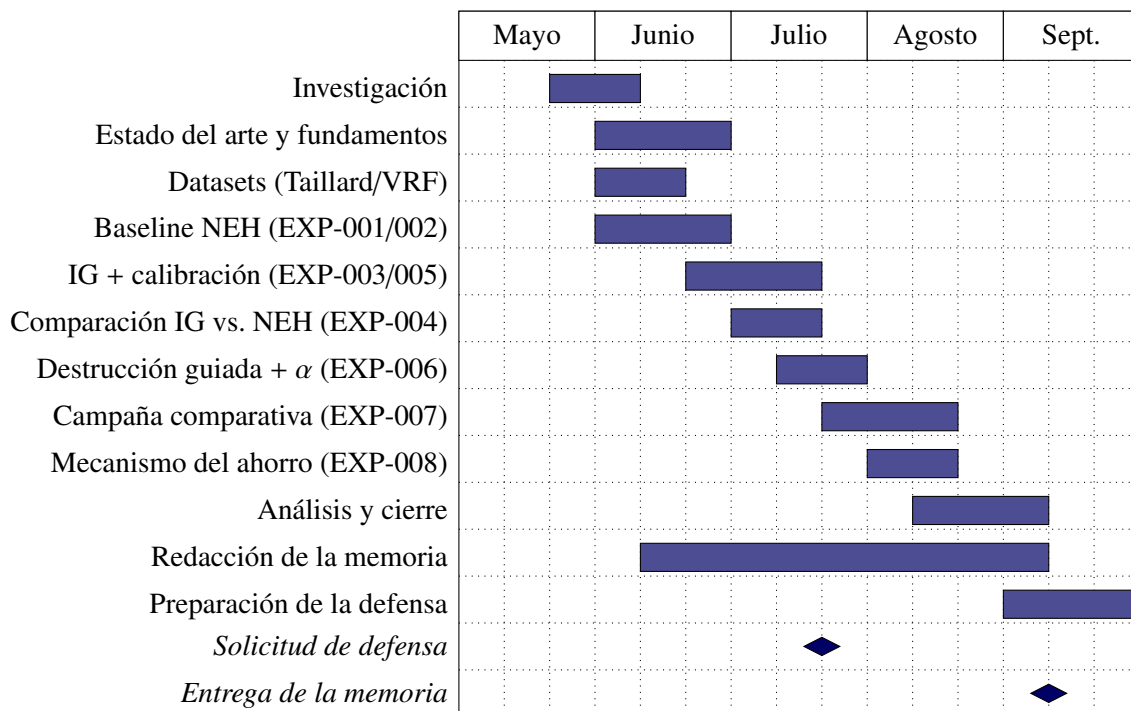


Fig. 7.1. Planificación temporal del proyecto (diagrama de Gantt). Las barras hasta julio reflejan el trabajo realizado; las posteriores, la planificación prevista. Los hitos marcan la solicitud de defensa (13–14 jul) y la entrega de la memoria (4–7 sep).

7.2. Presupuesto del TFG

El presupuesto recoge el coste de elaboración del trabajo. Al tratarse de un proyecto de investigación algorítmica realizado por una sola persona, los costes se agrupan en personal, hardware (amortización), software y costes indirectos.

7.2.1. Costes de personal

El TFG equivale a 12 ECTS; con la conversión de 25–30 horas por crédito, la dedicación del estudiante se sitúa en la banda de 300–360 horas, y se estima en 320 horas efectivas. A ellas se añade la dedicación del tutor en tutorías, revisión de los borradores y seguimiento, estimada en 20 horas a lo largo de los cinco meses del proyecto. La Tabla 7.1 recoge el desglose resultante.

Las tarifas horarias aplicadas (12 €/h para el estudiante y 38 €/h para el tutor) son una estimación propia, no tomada de ninguna tabla salarial publicada. Se sitúan en el orden de magnitud del coste horario de un ingeniero junior y de personal docente e investigador sénior en el mercado español, y se aplican de forma constante a todo el proyecto. Al tratarse de estimaciones, el valor informativo del presupuesto está en la estructura del gasto y en su magnitud relativa, no en la precisión de la cifra final.

Rol	Horas	€/hora	Total (€)
Estudiante (ingeniero junior)	320	12	3.840,00
Tutor (supervisión)	20	38	760,00
Subtotal personal			4.600,00

TABLA 7.1. COSTES DE PERSONAL.

7.2.2. Costes de hardware (amortización)

El desarrollo se realizó en un equipo informático convencional, sin cómputo en la nube. Parte de las campañas experimentales se ejecutó además en un servidor del laboratorio facilitado por la tutela (Sección 3.5.5), que se imputa por el tiempo que estuvo en uso. Se aplica una amortización lineal proporcional a la duración del proyecto, con una vida útil de cinco años (60 meses); los precios de adquisición son estimaciones de mercado para equipos de características equivalentes. El resultado se detalla en la Tabla 7.2.

Conviene subrayar que el servidor no es un requisito de la solución, sino un recurso que acortó los tiempos de espera de la experimentación: el algoritmo se ejecuta en el portátil sin ninguna dependencia de hardware especializado.

Equipo	Precio (€)	Vida útil	Uso (meses)	Amort. (€)
Portátil de desarrollo	2.500	60 meses	5	208,33
Servidor del laboratorio	1.800	60 meses	2	60,00
Subtotal hardware				268,33

TABLA 7.2. COSTES DE HARDWARE (AMORTIZACIÓN LINEAL A CINCO AÑOS).

7.2.3. Costes de software

El coste de software es nulo, como recoge la Tabla 7.3, porque todo el *stack* tecnológico empleado es libre y ninguna de sus piezas exige licencia de pago.

Software	Tipo	Coste (€)
Python, NumPy, Numba, matplotlib, pytest, ruff, black	Software libre	0
Editor / IDE	Libre o ya disponible	0
Subtotal software		0

TABLA 7.3. COSTES DE SOFTWARE.

Ese cero cobra sentido al compararlo con la alternativa. Los *solvers* comerciales de referencia ofrecen ediciones gratuitas, pero acotadas por el tamaño del modelo y restringidas a uso no comercial: la *Community Edition* de CPLEX admite hasta 1.000 variables y 1.000 restricciones [28], y la licencia reducida que acompaña a la distribución de Gurobi, hasta 2.000 variables y 2.000 restricciones lineales [29]. Un uso en producción obliga, por tanto, a contratar una licencia de pago. El nivel de entrada de CPLEX se publica desde 355 € mensuales por usuario autorizado (unos 4.260 € anuales) y está limitado expresamente a desarrollo, de modo que el despliegue en servidor se cotiza aparte [28]; Gurobi no publica tarifas y tramita todas sus licencias comerciales mediante solicitud de presupuesto [29].

La comparación da la medida del ahorro. Solo la suscripción de entrada de un único usuario durante un año equivale al 71 % del presupuesto total de este trabajo, sin cubrir el despliegue en producción, y el precio efectivo de ese despliegue no es conocible sin negociar con el proveedor. A la barrera económica se añade así una barrera de opacidad, que para una PYME sin departamento de compras especializado no es un obstáculo menor. Evitar por completo esa vía constituye uno de los argumentos centrales de viabilidad económica de la propuesta.

7.2.4. Costes indirectos y total

El resumen del presupuesto agrupa las partidas anteriores y añade dos conceptos que todavía no se han imputado. El primero son los gastos de electricidad y conectividad del puesto de trabajo, que se estiman en 63 € mensuales a lo largo de los cinco meses de

duración del proyecto y se tratan como coste directo. El segundo son los costes indirectos: gastos generales que el proyecto consume pero que no pueden atribuirse a una partida concreta, como el espacio de trabajo, su mantenimiento o la carga administrativa asociada. Al no ser medibles de forma individual, se estiman aplicando un 15 % sobre el total de los costes directos, porcentaje de uso habitual en proyectos de esta naturaleza. La Tabla 7.4 resume el conjunto.

Concepto	Coste (€)
Personal	4.600,00
Hardware (amortización)	268,33
Software	0,00
Electricidad y conectividad (63 €/mes × 5)	315,00
Subtotal costes directos	5.183,33
Costes indirectos (15 % sobre directos)	777,50
TOTAL del proyecto	5.960,83

TABLA 7.4. RESUMEN DE COSTES Y TOTAL DEL PROYECTO.

El presupuesto total del proyecto asciende, por tanto, a 5.960,83 €. No se recogen gastos de desplazamiento, de servicios de cómputo en la nube ni de licencias de software porque no llegaron a producirse: la experimentación se realizó íntegramente en un equipo propio y en el servidor del laboratorio, y todas las herramientas empleadas son libres.

El reparto confirma lo que anticipaba el planteamiento: el 77 % del presupuesto corresponde a la dedicación de personal, mientras que la infraestructura y el software no llegan al 5 % del total. Es la distribución esperable en un proyecto de investigación algorítmica y, al mismo tiempo, el argumento económico de la propuesta: replicar la solución en una PYME no requiere inversión en licencias ni en equipamiento especializado, solo el tiempo de quien la pone en marcha.

7.3. Impacto socio-económico

Más allá de su coste de elaboración, el trabajo tiene un impacto potencial que se analiza en cuatro dimensiones: económica, social, medioambiental y ética. El hilo conductor es la reducción de la barrera de entrada a la optimización de la producción para las PYMEs; a partir de ahí, cada apartado examina un tipo de efecto esperado. La dimensión medioambiental se trata de forma estrictamente cualitativa, coherente con la restricción del trabajo de no modelar el consumo energético.

7.3.1. Impacto económico

Para una PYME industrial, reducir el *makespan* significa completar la misma carga de trabajo en menos tiempo con la maquinaria ya instalada, aumentando el rendimiento

sin inversión adicional en equipos. El coste de adopción de la solución propuesta es bajo: software libre, ejecución en equipos convencionales (*plug-and-play*) y sin licencias comerciales ni personal especializado en optimización. Esto rebaja la barrera de entrada que hoy aparta a muchas PYMEs de las técnicas de optimización de la producción.

7.3.2. Impacto social

La propuesta democratiza el acceso a la optimización de la producción: acerca herramientas habitualmente reservadas a grandes empresas (con *solvers* comerciales e infraestructura de cómputo) al tejido de pequeñas y medianas empresas. Indirectamente, una planificación más eficiente puede mejorar la competitividad y la estabilidad del empleo en ese tejido industrial.

7.3.3. Impacto medioambiental (cualitativo)

Reducir el *makespan* y los *idle times* disminuye el tiempo que la maquinaria permanece encendida en espera, en línea con las prácticas de *Green Scheduling*. Este beneficio se presenta de forma estrictamente cualitativa y narrativa: el trabajo no modela el consumo energético ni cuantifica emisiones; la única métrica optimizada es el tiempo (C_{max}).

7.3.4. Impacto ético

El proyecto se desarrolla con software libre y resultados reproducibles (transparencia), no trata datos personales y declara el uso de IA generativa en el Anexo B. La herramienta es de apoyo a la decisión (propone una secuencia de producción), por lo que la responsabilidad operativa permanece en la organización que la utiliza.

7.4. Relación con los Objetivos de Desarrollo Sostenible

La Agenda 2030 enmarca el impacto del trabajo principalmente en tres objetivos:

- **ODS 9 — Industria, innovación e infraestructura:** el proyecto lleva técnicas de optimización avanzadas a procesos industriales de pequeña escala, fomentando la innovación accesible.
- **ODS 12 — Producción y consumo responsables:** un uso más eficiente de la capacidad productiva instalada (menos tiempo muerto) contribuye a un consumo de recursos más responsable.
- **ODS 8 — Trabajo decente y crecimiento económico:** la mejora de la productividad de las PYMEs tiene un efecto positivo potencial sobre su competitividad y, por extensión, sobre la calidad del empleo.

De forma narrativa, no cuantificada, la reducción de tiempos muertos se alinea también con el ODS 7 (energía asequible) y el ODS 13 (acción por el clima) a través de la eficiencia; se mencionan como derivada cualitativa, sin modelar consumo energético.

7.5. Síntesis del coste y del impacto esperado

El coste del TFG se concentra en la dedicación de personal, con un coste de software nulo gracias al *stack* libre. La solución no exige hardware especializado, ya que se ejecuta en un equipo convencional y todo el cómputo es en CPU; parte de las campañas experimentales se ejecutó en un servidor del laboratorio, pero solo para acortar los tiempos de espera, no porque el algoritmo lo requiera. Ambos factores prefiguran su viabilidad económica para PYMEs. El impacto esperado es positivo en lo económico (mayor rendimiento sin inversión), social (acceso democratizado) y, de forma cualitativa, medioambiental (eficiencia y *Green Scheduling*), con encaje claro en los ODS 9, 12 y 8.

BIBLIOGRAFÍA

- [1] M. R. Garey, D. S. Johnson y R. Sethi, “The complexity of flowshop and jobshop scheduling,” *Mathematics of Operations Research*, vol. 1, n.º 2, pp. 117-129, 1976. doi: [10.1287/moor.1.2.117](https://doi.org/10.1287/moor.1.2.117)
- [2] Gurobi Optimization, LLC. “Gurobi Optimizer Reference Manual,” Acceso: 16 de jul. de 2026. [En línea]. Disponible en: <https://www.gurobi.com>
- [3] IBM. “IBM ILOG CPLEX Optimization Studio,” Acceso: 16 de jul. de 2026. [En línea]. Disponible en: <https://www.ibm.com/products/ilog-cplex-optimization-studio>
- [4] M. Nawaz, E. E. Ensore e I. Ham, “A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem,” *Omega*, vol. 11, n.º 1, pp. 91-95, 1983. doi: [10.1016/0305-0483\(83\)90088-9](https://doi.org/10.1016/0305-0483(83)90088-9)
- [5] R. Ruiz y T. Stützle, “A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem,” *European Journal of Operational Research*, vol. 177, n.º 3, pp. 2033-2049, 2007. doi: [10.1016/j.ejor.2005.12.009](https://doi.org/10.1016/j.ejor.2005.12.009)
- [6] E. Vallada, R. Ruiz y J. M. Framinan, “New hard benchmark for flowshop scheduling problems minimising makespan,” *European Journal of Operational Research*, vol. 240, n.º 3, pp. 666-677, 2015. doi: [10.1016/j.ejor.2014.07.033](https://doi.org/10.1016/j.ejor.2014.07.033)
- [7] R. L. Graham, E. L. Lawler, J. K. Lenstra y A. H. G. Rinnooy Kan, “Optimization and approximation in deterministic sequencing and scheduling: a survey,” en *Annals of Discrete Mathematics*, vol. 5, 1979, pp. 287-326. doi: [10.1016/S0167-5060\(08\)70356-X](https://doi.org/10.1016/S0167-5060(08)70356-X)
- [8] S. M. Johnson, “Optimal two- and three-stage production schedules with setup times included,” *Naval Research Logistics Quarterly*, vol. 1, n.º 1, pp. 61-68, 1954. doi: [10.1002/nav.3800010110](https://doi.org/10.1002/nav.3800010110)
- [9] É. D. Taillard, “Some efficient heuristic methods for the flow shop sequencing problem,” *European Journal of Operational Research*, vol. 47, n.º 1, pp. 65-74, 1990. doi: [10.1016/0377-2217\(90\)90090-X](https://doi.org/10.1016/0377-2217(90)90090-X)
- [10] J. R. King y A. S. Spachis, “Heuristics for flow-shop scheduling,” *International Journal of Production Research*, vol. 18, n.º 3, pp. 345-357, 1980. doi: [10.1080/00207548008919673](https://doi.org/10.1080/00207548008919673)
- [11] E. Nowicki y C. Smutnicki, “A fast tabu search algorithm for the permutation flow-shop problem,” *European Journal of Operational Research*, vol. 91, n.º 1, pp. 160-175, 1996. doi: [10.1016/0377-2217\(95\)00037-2](https://doi.org/10.1016/0377-2217(95)00037-2)

- [12] J. Gmys, M. Mezmaz, N. Melab y D. Tuytens, "A computationally efficient Branch-and-Bound algorithm for the permutation flow-shop scheduling problem," *European Journal of Operational Research*, vol. 284, n.º 3, pp. 814-833, 2020. doi: [10.1016/j.ejor.2020.01.039](https://doi.org/10.1016/j.ejor.2020.01.039) [En línea]. Disponible en: <https://zenodo.org/records/3550553>
- [13] R. Ruiz y C. Maroto, "A comprehensive review and evaluation of permutation flowshop heuristics," *European Journal of Operational Research*, vol. 165, n.º 2, pp. 479-494, 2005. doi: [10.1016/j.ejor.2004.04.017](https://doi.org/10.1016/j.ejor.2004.04.017)
- [14] V. Fernandez-Viagas, R. Ruiz y J. M. Framinan, "A new vision of approximate methods for the permutation flowshop to minimise makespan: State-of-the-art and computational evaluation," *European Journal of Operational Research*, vol. 257, n.º 3, pp. 707-721, 2017. doi: [10.1016/j.ejor.2016.09.055](https://doi.org/10.1016/j.ejor.2016.09.055)
- [15] I. H. Osman y C. N. Potts, "Simulated annealing for permutation flow-shop scheduling," *Omega*, vol. 17, n.º 6, pp. 551-557, 1989. doi: [10.1016/0305-0483\(89\)90059-5](https://doi.org/10.1016/0305-0483(89)90059-5)
- [16] C. R. Reeves, "A genetic algorithm for flowshop sequencing," *Computers & Operations Research*, vol. 22, n.º 1, pp. 5-13, 1995. doi: [10.1016/0305-0548\(93\)E0014-K](https://doi.org/10.1016/0305-0548(93)E0014-K)
- [17] C. Rajendran y H. Ziegler, "Ant-colony algorithms for permutation flowshop scheduling to minimize makespan/total flowtime of jobs," *European Journal of Operational Research*, vol. 155, n.º 2, pp. 426-438, 2004. doi: [10.1016/S0377-2217\(02\)00908-6](https://doi.org/10.1016/S0377-2217(02)00908-6) [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0377221702009086>
- [18] R. Ruiz, Q.-K. Pan y B. Naderi, "Iterated Greedy methods for the distributed permutation flowshop scheduling problem," *Omega*, vol. 83, pp. 213-222, 2019. doi: [10.1016/j.omega.2018.03.004](https://doi.org/10.1016/j.omega.2018.03.004)
- [19] J. Dubois-Lacoste, F. Pagnozzi y T. Stützle, "An iterated greedy algorithm with optimization of partial solutions for the makespan permutation flowshop problem," *Computers & Operations Research*, vol. 81, pp. 160-166, 2017. doi: [10.1016/j.cor.2016.12.021](https://doi.org/10.1016/j.cor.2016.12.021)
- [20] Q.-K. Pan y R. Ruiz, "An effective iterated greedy algorithm for the mixed no-idle permutation flowshop scheduling problem," *Omega*, vol. 44, pp. 41-50, 2014. doi: [10.1016/j.omega.2013.10.002](https://doi.org/10.1016/j.omega.2013.10.002)
- [21] V. Fernandez-Viagas y J. M. Framinan, "On insertion tie-breaking rules in heuristics for the permutation flowshop scheduling problem," *Computers & Operations Research*, vol. 45, pp. 60-67, 2014. doi: [10.1016/j.cor.2013.12.012](https://doi.org/10.1016/j.cor.2013.12.012)

- [22] M. Karimi-Mamaghan, M. Mohammadi, B. Pasdeloup y P. Meyer, “Learning to select operators in meta-heuristics: An integration of Q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem,” *European Journal of Operational Research*, vol. 304, n.º 3, pp. 1296-1330, 2023. doi: [10.1016/j.ejor.2022.03.054](https://doi.org/10.1016/j.ejor.2022.03.054)
- [23] K.-C. Ying y S.-W. Lin, “Reinforcement learning iterated greedy algorithm for distributed assembly permutation flowshop scheduling problems,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, n.º 8, pp. 11 123-11 138, 2023. doi: [10.1007/s12652-022-04392-w](https://doi.org/10.1007/s12652-022-04392-w) [En línea]. Disponible en: <https://link.springer.com/article/10.1007/s12652-022-04392-w>
- [24] W. Liu, Y. Jin y M. Price, “A new Nawaz–Enscore–Ham-based heuristic for permutation flow-shop problems with bicriteria of makespan and machine idle time,” *Engineering Optimization*, vol. 48, n.º 10, pp. 1808-1822, 2016. doi: [10.1080/0305215X.2016.1141202](https://doi.org/10.1080/0305215X.2016.1141202) [En línea]. Disponible en: <https://www.tandfonline.com/doi/full/10.1080/0305215X.2016.1141202>
- [25] Q. Ji, Y. Han, Y. Wang, B. Zhang y K. Gao, “A Penalty Groups-Assisted Iterated Greedy Integrating Idle Time Insertion: Solving the Hybrid Flow Shop Group Scheduling with Delivery Time Windows,” *Complex System Modeling and Simulation*, vol. 4, n.º 2, pp. 137-165, 2024. doi: [10.23919/CSMS.2024.0005](https://doi.org/10.23919/CSMS.2024.0005)
- [26] Y. Li et al., “Graph Knowledge-Enhanced Iterated Greedy Algorithm for Hybrid Flowshop Scheduling Problem,” *Mathematics*, vol. 13, n.º 15, p. 2401, 2025. doi: [10.3390/math13152401](https://doi.org/10.3390/math13152401) [En línea]. Disponible en: <https://www.mdpi.com/2227-7390/13/15/2401>
- [27] É. D. Taillard, “Benchmarks for basic scheduling problems,” *European Journal of Operational Research*, vol. 64, n.º 2, pp. 278-285, 1993. doi: [10.1016/0377-2217\(93\)90182-M](https://doi.org/10.1016/0377-2217(93)90182-M)
- [28] IBM. “IBM ILOG CPLEX Optimization Studio—Precios,” Acceso: 2 de sep. de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/products/ilog-cplex-optimization-studio/pricing>
- [29] Gurobi Optimization, LLC. “Pricing and Licensing,” Acceso: 2 de sep. de 2026. [En línea]. Disponible en: <https://www.gurobi.com/product/pricing-and-licensing>

ANEXO A. GUÍA DE REPRODUCCIÓN (PRODUCTO MÍNIMO VIABLE)

Este anexo resume los pasos para clonar el repositorio y poner en marcha la solución en un equipo convencional, materializando el objetivo 5 (producto mínimo viable). El repositorio contiene el paquete algorítmico y su batería de pruebas, las instancias de *benchmark*, la evidencia cruda de las campañas experimentales y esta memoria. Lo que sigue es un resumen operativo; la guía detallada y actualizada acompaña al código en el repositorio (Code/README.md y Code/EXECUTION_GUIDE.md). El repositorio se aloja en el grupo CAOS de la UC3M¹.

Requisitos. Python 3.12 o superior y git. No se requieren *solvers* comerciales ni infraestructura de alto rendimiento.

1. Obtener el código y crear el entorno. El nombre del repositorio se asigna a una variable solo para que la orden de clonado quepa en una línea.

```
REPO=CAOSResearch/iterated_greedy_permutation_flow_shop
git clone https://github.com/$REPO pfsp-ig
cd pfsp-ig
python3.12 -m venv .venv
source .venv/bin/activate          # macOS / Linux
```

2. Instalar las dependencias y el paquete.

```
pip install -r Code/requirements.txt
pip install -e Code/
python -c "import pfsp; print('pfsp ok')"
```

3. Ejecutar los algoritmos sobre una instancia.

```
# Baseline constructivo NEH
pfsp-neh --instance Code/data/taillard/tai20_5_0.fsp

# Iterated Greedy (semilla fija, reproducible)
pfsp-ig --instance Code/data/taillard/tai20_5_0.fsp --seed 42
```

4. Resolver una planta propia. El paso anterior emplea instancias de *benchmark*, pero el objetivo del producto mínimo viable es que una PYME pueda resolver su propio caso. Para ello basta describir la planta en un fichero de texto con extensión *.fsp*: una

¹URL del repositorio: https://github.com/CAOSResearch/iterated_greedy_permutation_flow_shop; permanece privado hasta la fecha de defensa.

línea por máquina, en el orden en que los trabajos las recorren, con un entero no negativo por trabajo que indique su tiempo de proceso en esa máquina. Las dimensiones se deducen de la propia matriz, de modo que no hace falta cabecera alguna. El contenido siguiente describe una planta de tres máquinas y seis trabajos:

```
54 83 15 71 77 36
79  3 11 99 56 70
16 89 49 15 89 45
```

Guardado como `mi_planta.fsp`, se resuelve con la línea base y con el algoritmo propuesto, incluido su operador de destrucción guiada:

```
pfsp-neh --instance mi_planta.fsp

pfsp-ig --instance mi_planta.fsp --seed 42 \
    --destruction idle-rcl --alpha 0.30
```

Ambas invocaciones imprimen la secuencia de trabajos obtenida y su *makespan*, que es el dato de interés operativo: el orden en que conviene introducir los pedidos en la línea. La columna `rpd` aparece en cambio como `n/a`, porque la RPD se calcula frente al mejor *makespan* publicado de la instancia y ese valor de referencia solo existe para las instancias de *benchmark*. No es necesario convertir nada si se parte de instancias ya publicadas: el lector reconoce los formatos originales de Taillard y de VRF a partir del nombre del fichero, con una comprobación de su contenido como respaldo (`Code/pfsp/io/loader.py`), y los normaliza a la misma representación interna.

5. Consultar los resultados de la experimentación. El repositorio incluye la evidencia cruda de las campañas del Capítulo 4, en los dos entornos de ejecución empleados: `Code/results/` (equipo portátil) y `Code/results-server/` (servidor del laboratorio). En ambos, `raw/` guarda una fila por ejecución individual (no medias), con un fichero por algoritmo (`NEH.csv`, `IG.csv` y los de las dos variantes guiadas), y `calibration/` recoge los barridos de presupuesto y del parámetro α . Las carpetas restantes son propias de un solo entorno: `instrumentation/`, con la medición del mecanismo, solo existe en el servidor, que es donde se ejecutó esa campaña; y `summary/` y `figures/`, que son productos derivados, solo en el equipo portátil. La tabla de `summary/` se deriva del crudo mediante el agregador del paquete y puede regenerarse en cualquier momento, de modo que ninguna cifra de la memoria se escribe a mano. Para reproducir una campaña completa, el motor reanudable (`Code/pfsp/experiments/runner.py`) admite el conjunto de semillas, la configuración del algoritmo y la ejecución por tramos, retomando sin duplicar filas.

6. Verificar el proyecto. Las herramientas de verificación no forman parte de las dependencias de ejecución, así que se instalan aparte.

```
pip install -r Code/requirements-dev.txt
```

```
pytest Code/tests      # pruebas (unitarias y basadas en propiedades)
ruff check Code        # linter
black --check Code     # formateo
```

Cada ejecución nueva añade su registro crudo a `Code/results/raw/` y escribe un *manifest* con los parámetros y el entorno capturado, que es el vínculo entre una cifra y su contexto de ejecución. Los *manifests* de las campañas ya publicadas —uno por ejecución, varios miles de ficheros— no se distribuyen por volumen; sí lo hace el registro crudo, que es la evidencia irremplazable. Esa separación entre crudo conservado y resumen derivado es la que sostiene la trazabilidad y la reproducibilidad descritas en el Capítulo 3.

ANEXO B. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

He usado IAG en mi TFG: **SÍ**.

Parte 1: declaración sobre comportamiento legal, ético y responsable

- *Datos de carácter confidencial.* **NO**, no he facilitado datos de carácter confidencial a herramientas de IAG. El trabajo se apoya en conjuntos de instancias académicos de dominio público (Taillard y VRF) y no maneja información confidencial de ninguna organización.
- *Materiales protegidos por derechos de autoría.* **SÍ**, he utilizado materiales protegidos sin autorización expresa de sus titulares, por ajustarse a las excepciones o límites que permite la ley: el *uso de fragmentos con fines de investigación* (derecho de cita) en la literatura científica consultada, y la *obra en dominio público o licenciada* en los conjuntos de instancias y la documentación técnica.
- *Datos de carácter personal.* **NO**, no he facilitado datos de carácter personal a herramientas de IAG. El trabajo no trata datos personales, tal como se razona en el capítulo de marco regulador.
- *Términos de uso y principios éticos.* **SÍ**, la utilización de las herramientas ha respetado sus términos de uso y los principios éticos esenciales. No se han orientado de manera maliciosa a producir una impresión contraria a la realidad de los resultados obtenidos, ni a suplantar el trabajo propio, ni a nada que pudiera resultar en un perjuicio para las personas.

Parte 2: declaración de uso técnico

Declaro haber hecho uso de los sistemas de IAG Kiro —en modo agente sobre el repositorio del proyecto, con los modelos Claude Opus 4.6 y Claude Opus 5— y Gemini, en sus versiones 2.5 Pro y 1.5 Pro (modo *Deep Research*), para los usos que se detallan a continuación.

Documentación y redacción

- *Soporte a la reflexión y análisis de alternativas.* Exploración del espacio de diseño del operador propuesto: se solicitó un análisis de las palancas de diseño disponibles y de operadores candidatos, junto con sus riesgos previsibles.
- *Revisión y reescritura de texto redactado previamente.* Corrección y mejora de la redacción sobre texto ya escrito por el autor, conservando sus ideas y su contenido, y

propuestas de ordenación sobre un índice previamente definido por él. La estructura del documento es una decisión propia y se acordó con el tutor en las reuniones de seguimiento.

- *Búsqueda de información y respuesta a preguntas concretas.* Consultas sobre normativa de la titulación, formato del documento y aspectos técnicos de LaTeX.
- *Búsqueda de bibliografía.* Se emplearon informes de investigación de Gemini para localizar literatura sobre el problema y sobre variantes del algoritmo. Toda referencia obtenida por esta vía se verificó después de forma independiente contra Crossref y el DOI original, y las que no pudieron confirmarse se descartaron y no figuran en la bibliografía.
- *Resumen de la bibliografía consultada.* Elaboración de fichas de lectura de los artículos, con sus metadatos y su aportación, a partir de fuentes ya verificadas.

Desarrollo de contenido específico

- *Asistencia en el desarrollo de código.* El desarrollo partió de código propio del autor y, sobre esa base, se recurrió a la asistencia de IAG para extenderlo, depurarlo y completarlo, revisando e integrando el resultado bajo su supervisión. Las decisiones de diseño —la arquitectura del paquete, la elección del operador propuesto, el *stack* tecnológico y el diseño experimental— fueron del autor.
- *Procesos de optimización.* Asistencia en la mejora del rendimiento del cálculo del *makespan* y de la evaluación de inserciones, así como en dotar a los *runners* de reanudación por instancia para poder ejecutar campañas largas por tramos.
- *Tratamiento de datos.* Asistencia en el desarrollo de los *scripts* que agregan los resultados crudos de las ejecuciones, calculan la métrica de comparación y aplican los contrastes estadísticos con los que se comparan los operadores. El tratamiento es automático y parte siempre de los ficheros de resultados que generaron las ejecuciones reales: ningún dato se introdujo ni se ajustó a mano.
- *Generación de esquemas y figuras.* Las figuras del documento no se generaron con un modelo de imagen: se produjeron mediante *scripts* deterministas escritos con asistencia de IAG, que las construyen a partir de los datos experimentales reales o de la descripción conceptual del algoritmo, de modo que son reproducibles ejecutando el repositorio.

Parte 3: reflexión sobre utilidad

En cuanto a las fortalezas, el uso de estas herramientas me permitió profundizar más en el problema sin que ello penalizara el tiempo de desarrollo. El ejemplo más claro es

el volumen de experimentación que llegué a sostener: sin esa ayuda difícilmente habría ejecutado y analizado las ocho campañas con las que justifico los argumentos de este trabajo y sostengo su hipótesis. También me sirvieron para contrastar ideas con enfoques científicos que en muchos casos desconocía de partida.

Entre las debilidades, identifiqué que estas herramientas producen con facilidad afirmaciones plausibles pero imprecisas. Eso me obligó a mantener una rutina de verificación constante: comprobar cada afirmación técnica contra el código y cada referencia contra su fuente original. En varias ocasiones propusieron referencias inexistentes que hubo que descartar, de modo que esa comprobación no fue un trámite, sino una parte real del trabajo.

Sobre su efecto en mi aprendizaje, considero que ha sido importante, aunque no exactamente en la dirección que esperaba. Me permitió profundizar en el tema sin dedicar incontables horas a la búsqueda manual de fuentes, si bien ese ahorro se compensó en parte con el tiempo invertido en verificar lo que proponían. Lo que me llevo de esta experiencia es que, correctamente configuradas, estas herramientas funcionan como acompañamiento: ayudan a entender los conceptos y cuestionan aproximaciones que no siempre son las más efectivas, lo que ha hecho que el trabajo se desarrolle de forma más rigurosa y con mejores fundamentos teóricos. El criterio para decidir qué aceptar y qué descartar, en cambio, ha tenido que ser mío.